

Decoradores de Python

Un decorador es un patrón de diseño en Python que permite a un usuario añadir nueva funcionalidad a un objeto existente sin modificar su estructura. Los decoradores generalmente se llaman antes de la definición de una función que desea decorar.

Creación de decoradores

Para crear una función de decorador, necesitamos una función externa con una función de envoltura interna.

Ejemplo:

```
# Normal function

def greeting():

    return 'Welcome to Python'

# function decoradora

def uppercase_decorator(function):

    def wrapper():

        func = function()

        make_uppercase = func.upper()

        return make_uppercase

    return wrapper

# Decoradora de normal function

g = uppercase_decorator(greeting)

print(g())      # WELCOME TO PYTHON
```

Parte 1: Descripción línea por línea del código

```
def greeting():

    return 'Welcome to Python'
```

`def greeting():` define una función ordinaria llamada `greeting` sin parámetros. `return 'Welcome to Python'` hace que devuelva ese string exacto cada vez que se la invoque.

```
def uppercase_decorator(function):
```

```
    def wrapper():
```

```
        func = function()
```

```
        make_uppercase = func.upper()
```

```
        return make_uppercase
```

```
    return wrapper
```

`def uppercase_decorator(function):` define la función decoradora, que recibe otra función como argumento (en Python las funciones son objetos de primera clase, por eso pueden pasarse como parámetros).

`def wrapper():` define una función interna anidada —el "envoltorio"— que es la que ejecutará la lógica aumentada. `func = function()` llama a la función original que fue recibida como argumento y guarda su resultado (el string). `make_uppercase = func.upper()` aplica el método `.upper()` sobre ese string, transformándolo a mayúsculas. `return make_uppercase` retorna el resultado ya transformado. `return wrapper` retorna el objeto función `wrapper` sin ejecutarlo, para que quien llame al decorador reciba la función envuelta.

```
g = uppercase_decorator(greeting)
```

```
print(g())
```

`g = uppercase_decorator(greeting)` llama al decorador pasándole `greeting` como argumento, y guarda en `g` el objeto función `wrapper` que se retornó.

`print(g())` ejecuta `wrapper()`, que internamente llama a `greeting()`, convierte el resultado a mayúsculas, y lo imprime: **WELCOME TO PYTHON**.

Parte 2: Análisis de la ejecución paso a paso (nivel avanzado)

Aquí se detalla lo que ocurre en la memoria de Python en cada frame de ejecución, tal como lo muestra el visualizador del PDF.

Steps 2–3: Definición de funciones en el Global Frame

Cuando Python interpreta **def greeting()** y **def uppercase_decorator(function)**, no *ejecuta* el cuerpo de ninguna. Simplemente crea dos objetos función en el heap y enlaza sus nombres en el **Global Frame**.

En el visualizador se ven dos entradas:

greeting → **function greeting()**

uppercase_decorator → **function uppercase_decorator(function).**

Esto es fundamental: en Python, **def** es una sentencia ejecutable que crea un objeto y lo asigna, no una declaración estática como en lenguajes compilados.

Python 3.11
[known limitations](#)

```
1 # Normal function
2 def greeting():
3     return 'Welcome to Python'
4 def uppercase_decorator(function):
5     def wrapper():
6         func = function()
7         make_uppercase = func.upper()
8         return make_uppercase
9     return wrapper
10 g = uppercase_decorator(greeting)
11 print(g())          # WELCOME TO PYTHON
```

[Edit this code](#)

→ line that just executed
→ next line to execute

Step 2 of 16

Print output (drag lower right corner to resize)

Frames: Global frame (greeting) → Objects: function greeting()

Python 3.11
[known limitations](#)

```
1 # Normal function
2 def greeting():
3     return 'Welcome to Python'
4 def uppercase_decorator(function):
5     def wrapper():
6         func = function()
7         make_uppercase = func.upper()
8         return make_uppercase
9     return wrapper
10 g = uppercase_decorator(greeting)
11 print(g())          # WELCOME TO PYTHON
```

[Edit this code](#)

→ line that just executed
→ next line to execute

Step 3 of 16

Print output (drag lower right corner to resize)

Frames: Global frame (greeting, uppercase_decorator) → Objects: function greeting(), function uppercase_decorator(function)

Steps 4–5: Llamada a **uppercase_decorator(greeting) — se crea el frame f1**

Al ejecutar `g = uppercase_decorator(greeting)`, Python abre un nuevo frame de activación (f1) para `uppercase_decorator`. El parámetro `function` queda enlazado al objeto `greeting()` del heap.

Dentro de ese frame, Python encuentra `def wrapper()`: y ejecuta esa sentencia `def`, lo cual crea un *nuevo* objeto función en el heap: `wrapper()` [`parent=f1`].

La anotación `parent=f1` es clave en Python: significa que `wrapper` tiene una referencia al frame donde fue creada (el *closure*). Ese mecanismo es lo que permite que `wrapper` acceda a `function` incluso después de que `f1` sea destruido.

The image displays two screenshots of the Python Tutor debugger, illustrating the execution of a decorator function.

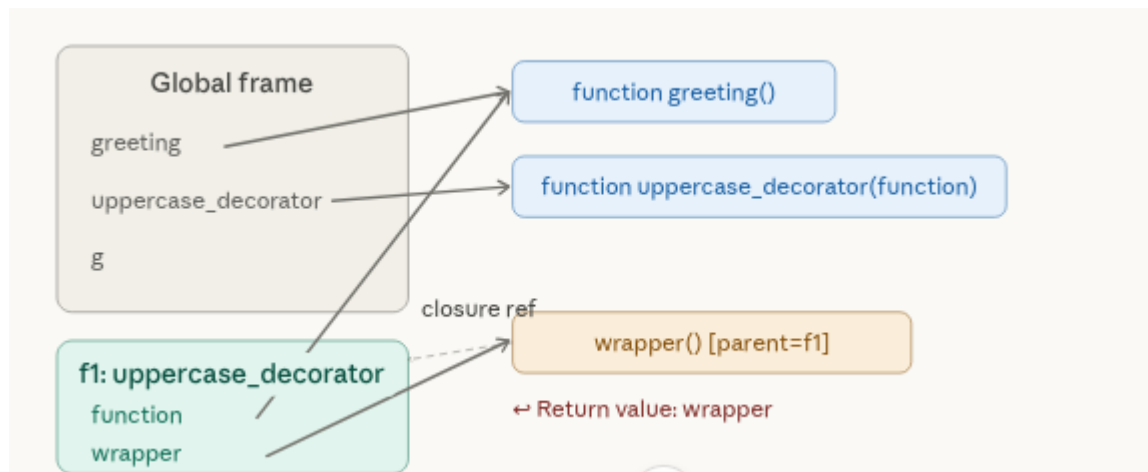
Top Screenshot (Step 4 of 16):

- Code:** The code shows the definition of `uppercase_decorator` and its call with `greeting`. Line 10, `g = uppercase_decorator(greeting)`, is the line that just executed.
- Frames:** The Global frame is active. It contains `greeting` and `uppercase_decorator`.
- Objects:** The `function` object `uppercase_decorator(function)` is created. The `function` parameter is linked to the `greeting` object in the Global frame.

Bottom Screenshot (Step 6 of 16):

- Code:** The code is the same, but now line 9, `return wrapper`, is the line that just executed.
- Frames:** A new frame, `f1: uppercase_decorator`, is created. It contains `function` and `wrapper`.
- Objects:** A new `function` object `wrapper() [parent=f1]` is created. The `function` parameter is linked to the `function` object in the `f1: uppercase_decorator` frame.

Steps 6–7: `uppercase_decorator` retorna `wrapper`



La sentencia **return wrapper** hace que `f1` devuelva el objeto función **wrapper** (no su resultado, sino la función misma). El frame `f1` queda marcado con un "Return value" apuntando a **wrapper**. Aquí está la esencia del patrón decorador: la función externa no ejecuta la lógica, la *empaqueta* y la devuelve diferida.

Python 3.11
[known limitations](#)

```

1 # Normal function
2 def greeting():
3     return 'Welcome to Python'
4 def uppercase_decorator(function):
5     def wrapper():
6         func = function()
7         make_uppercase = func.upper()
8         return make_uppercase
9     return wrapper
10 g = uppercase_decorator(greeting)
11 print(g())          # WELCOME TO PYTHON

```

[Edit this code](#)

→ line that just executed
→ next line to execute

<< First < Prev Next > Last >>

Step 7 of 16

Print output (drag lower right corner to resize)

Frames	Objects
Global frame	function <code>greeting()</code>
	function <code>uppercase_decorator(function)</code>
f1: uppercase_decorator	function <code>wrapper() [parent=f1]</code>
	Return value

Step 8: `g` queda enlazado a **wrapper** en el Global Frame

Cuando `f1` retorna, Python destruye ese frame de activación. Pero **wrapper** sigue vivo en el heap porque `g` lo referencia desde el **Global Frame**. Y como **wrapper** carga la referencia **parent=f1**, puede seguir accediendo a la variable **function** que vivía en `f1` —esto es el **closure**: una función que captura el entorno donde fue definida aunque ese entorno ya no exista en la pila de llamadas.

Python 3.11
[known limitations](#)

```
1 # Normal function
2 def greeting():
3     return 'Welcome to Python'
4 def uppercase_decorator(function):
5     def wrapper():
6         func = function()
7         make_uppercase = func.upper()
8         return make_uppercase
9     return wrapper
10 g = uppercase_decorator(greeting)
11 print(g()) # WELCOME TO PYTHON
```

→ line that just executed

→ next line to execute

Step 8 of 16

Print output (drag lower right corner to resize)

Frames

Objects

Global frame

greeting

uppercase_decorator

g

f1: uppercase_decorator

function

wrapper

Return value

function

greeting()

function

uppercase_decorator(function)

function

wrapper() [parent=f1]

Steps 9–10: Se abre el frame de **wrapper**

Al ejecutar **g()**, Python abre un nuevo frame de activación para **wrapper**. Dentro, se ejecuta **func = function()**: Python busca **function** primero en el frame local de **wrapper** (no existe), luego sube por la cadena de closures hasta encontrarla en el entorno capturado de **f1**. Llama a **greeting()**, que a su vez abre su propio frame brevemente. **greeting** retorna "Welcome to Python" y ese valor queda en **func**.

Python 3.11
[known limitations](#)

```
1 # Normal function
2 def greeting():
3     return 'Welcome to Python'
4 def uppercase_decorator(function):
5     def wrapper():
6         func = function()
7         make_uppercase = func.upper()
8         return make_uppercase
9     return wrapper
10 g = uppercase_decorator(greeting)
11 print(g()) # WELCOME TO PYTHON
```

→ line that just executed

→ next line to execute

Step 9 of 16

Print output (drag lower right corner to resize)

Frames

Objects

Global frame

greeting

uppercase_decorator

g

f1: uppercase_decorator

function

wrapper

Return value

function

greeting()

function

uppercase_decorator(function)

function

wrapper() [parent=f1]

wrapper [parent=f1]

Python 3.11
[known limitations](#)

```
1 # Normal function
2 def greeting():
3     return 'Welcome to Python'
4 def uppercase_decorator(function):
5     def wrapper():
6         func = function()
7         make_uppercase = func.upper()
8         return make_uppercase
9     return wrapper
10 g = uppercase_decorator(greeting)
11 print(g()) # WELCOME TO PYTHON
```

[Edit this code](#)

→ line that just executed
→ next line to execute

<< First < Prev Next > Last >>

Step 10 of 16

Print output (drag lower right corner to resize)

Frames

Objects

Global frame

greeting

uppercase_decorator

g

f1: uppercase_decorator

function

wrapper

Return value

function greeting()

function uppercase_decorator(function)

function wrapper() [parent=f1]

wrapper [parent=f1]

Python 3.11
[known limitations](#)

```
1 # Normal function
2 def greeting():
3     return 'Welcome to Python'
4 def uppercase_decorator(function):
5     def wrapper():
6         func = function()
7         make_uppercase = func.upper()
8         return make_uppercase
9     return wrapper
10 g = uppercase_decorator(greeting)
11 print(g()) # WELCOME TO PYTHON
```

[Edit this code](#)

→ line that just executed
→ next line to execute

<< First < Prev Next > Last >>

Step 11 of 16

Print output (drag lower right corner to resize)

Frames

Objects

Global frame

greeting

uppercase_decorator

g

f1: uppercase_decorator

function

wrapper

Return value

function greeting()

function uppercase_decorator(function)

function wrapper() [parent=f1]

wrapper [parent=f1]

greeting

Python 3.11
[known limitations](#)

```
1 # Normal function
2 def greeting():
3     return 'Welcome to Python'
4 def uppercase_decorator(function):
5     def wrapper():
6         func = function()
7         make_uppercase = func.upper()
8         return make_uppercase
9     return wrapper
10 g = uppercase_decorator(greeting)
11 print(g()) # WELCOME TO PYTHON
```

[Edit this code](#)

→ line that just executed
→ next line to execute

<< First < Prev Next > Last >>

Step 12 of 16

Print output (drag lower right corner to resize)

Frames

Objects

Global frame

greeting

uppercase_decorator

g

f1: uppercase_decorator

function

wrapper

Return value

function greeting()

function uppercase_decorator(function)

function wrapper() [parent=f1]

wrapper [parent=f1]

greeting

Python 3.11
[known limitations](#)

```

1 # Normal function
2 def greeting():
3     return 'Welcome to Python'
4 def uppercase_decorator(function):
5     def wrapper():
6         func = function()
7         make_uppercase = func.upper()
8         return make_uppercase
9     return wrapper
10 g = uppercase_decorator(greeting)
11 print(g())          # WELCOME TO PYTHON

```

[Edit this code](#)

→ line that just executed
→ next line to execute

<< First

< Prev

Next >

Last >>

Step 13 of 16

NEW: teachers get [free access](#) to ad-free/AI-free mode

Print output (drag lower right corner to resize)

Frames

Global frame

- greeting
- uppercase_decorator
- g

f1: uppercase_decorator

- function
- wrapper
- Return value

Objects

- function greeting()
- function uppercase_decorator(function)
- function wrapper() [parent=f1]

wrapper [parent=f1]

greeting	"Welcome to Python"
Return value	

Python 3.11
[known limitations](#)

```

1 # Normal function
2 def greeting():
3     return 'Welcome to Python'
4 def uppercase_decorator(function):
5     def wrapper():
6         func = function()
7         make_uppercase = func.upper()
8         return make_uppercase
9     return wrapper
10 g = uppercase_decorator(greeting)
11 print(g())          # WELCOME TO PYTHON

```

[Edit this code](#)

→ line that just executed
→ next line to execute

<< First

< Prev

Next >

Last >>

Step 14 of 16

NEW: teachers get [free access](#) to ad-free/AI-free mode

Print output (drag lower right corner to resize)

Frames

Global frame

- greeting
- uppercase_decorator
- g

f1: uppercase_decorator

- function
- wrapper
- Return value

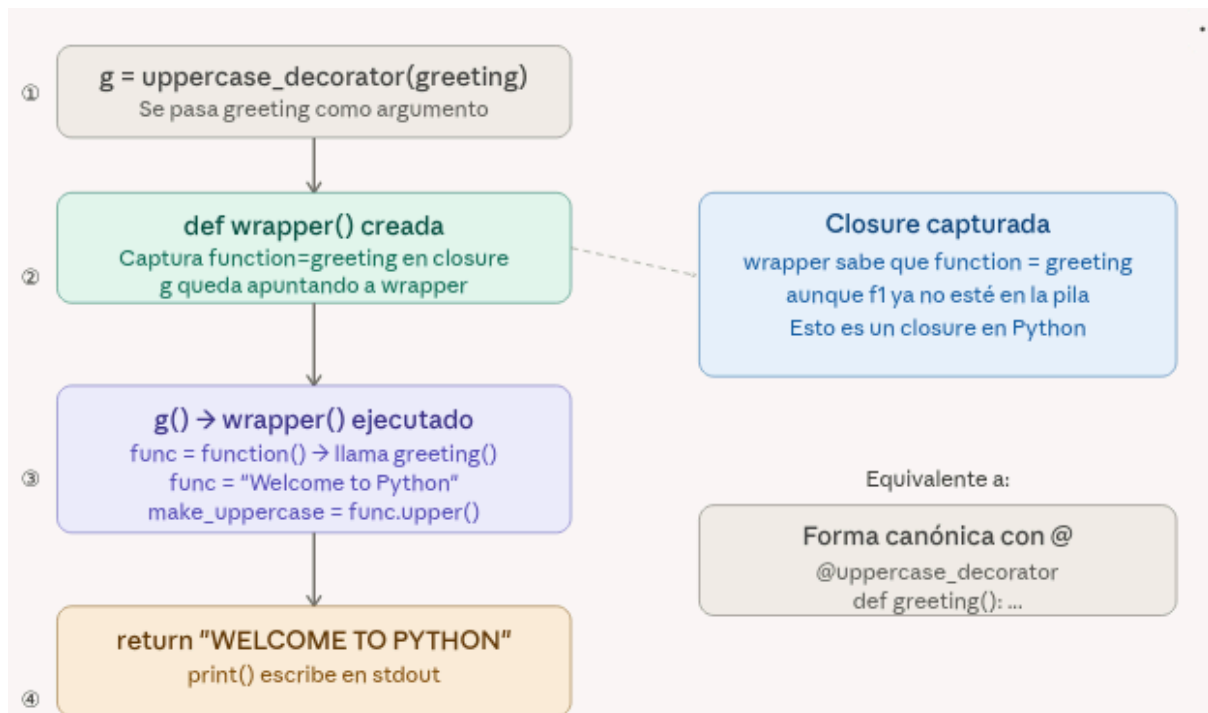
Objects

- function greeting()
- function uppercase_decorator(function)
- function wrapper() [parent=f1]

wrapper [parent=f1]

func	"Welcome to Python"
------	---------------------

Steps 11–16: Transformación y retorno final



`make_uppercase = func.upper()` invoca el método `upper()` del objeto string `"Welcome to Python"`, produciendo `"WELCOME TO PYTHON"`. `return` `make_uppercase` hace que el frame de `wrapper` muestre ese valor como Return value. `print()` recibe ese string y lo escribe en `stdout`.

Python 3.11
[known limitations](#)

```

1 # Normal function
2 def greeting():
3     return 'Welcome to Python'
4 def uppercase_decorator(function):
5     def wrapper():
6         func = function()
7         make_uppercase = func.upper()
8         return make_uppercase
9     return wrapper
10 g = uppercase_decorator(greeting)
11 print(g()) # WELCOME TO PYTHON

```

[Edit this code](#)

→ line that just executed
→ next line to execute

Step 15 of 16

NEW: teachers get [free access](#) to ad-free/AI-free mode

Print output (drag lower right corner to resize)

Frames	Objects
Global frame	function greeting()
uppercase_decorator	function uppercase_decorator(function)
g	function wrapper() [parent=f1]
f1: uppercase_decorator	function wrapper
	Return value

wrapper [parent=f1]

func	"Welcome to Python"
make_uppercase	"WELCOME TO PYTHON"

Python 3.11

known limitations

```

1 # Normal function
2 def greeting():
3     return 'Welcome to Python'
4 def uppercase_decorator(function):
5     def wrapper():
6         func = function()
7         make_uppercase = func.upper()
8         return make_uppercase
9     return wrapper
10 g = uppercase_decorator(greeting)
11 print(g()) # WELCOME TO PYTHON

```

→ line that just executed

→ next line to execute

Step 16 of 16

NEW: teachers get [free access](#) to ad-free/AI-free mode

Print output (drag lower right corner to resize)

Global frame

greeting

uppercase_decorator

g

f1: uppercase_decorator

function

wrapper

Return value

function

greeting()

function

uppercase_decorator(function)

function

wrapper() [parent=f1]

wrapper [parent=f1]

func	"Welcome to Python"
make_uppercase	"WELCOME TO PYTHON"
Return value	"WELCOME TO PYTHON"

Let us implement the example above with a decorator

"""This decorator function is a higher order function

that takes a function as a parameter"""

def uppercase_decorator(function):

def wrapper():

func = function()

make_uppercase = func.upper()

return make_uppercase

return wrapper

@uppercase_decorator

def greeting():

return 'Welcome to Python'

print(greeting()) # WELCOME TO PYTHON

En cambio, Python le permite usar decoradores de una manera más simple con el Símbolo, a veces llamado la [sintaxis del pastel](#). El siguiente ejemplo hace exactamente lo mismo que el primer ejemplo de decorador:

En este código, la función **decorator** es la que está decorando a la función **greeting**.

Para entenderlo claramente en términos de roles:

- **El Decorador (`uppercase_decorator`)**: Es la función "envoltorio". Su trabajo es recibir una función, instanciar una nueva función interna (`wrapper`) que le añade comportamiento extra (los `print` antes y después), y devolver esa nueva función.
- **La Función Decorada (`greeting`)**: Es la función objetivo. Es la función original que se pasa como argumento (`func`) al decorador para que su comportamiento sea modificado.

Cómo pasarlo a la sintaxis @

El símbolo @ en Python es simplemente "azúcar sintáctico" (syntactic sugar) que hace exactamente esa reasignación por detrás, para que no tengas que escribirla manualmente al final de tu código.

La regla es: **El símbolo @ va seguido del nombre de la función decoradora, y se coloca justo en la línea anterior a la definición de la función que quieres decorar.**

```
def greeting():
    return 'Welcome to Python'
def uppercase_decorator(function):
    def wrapper():
        func = function()
        make_uppercase = func.upper()
        return make_uppercase
    return wrapper
g = uppercase_decorator(greeting)
print(g())      # WELCOME TO PYTHON
```

Así que, @uppercase_decorator Es sólo una forma más corta de decir
g = uppercase_decorator(greeting). Es la forma de aplicar un decorador a una función.

@uppercase_decorator ⇒ g = uppercase_decorator(greeting)

Síntesis conceptual (nivel avanzado)

Funciones como objetos de primera clase. greeting no es solo un nombre —es una referencia a un objeto en el heap que puede pasarse, devolverse y almacenarse como cualquier otro valor.

Closures y el LEGB Rule. Cuando wrapper busca function, Python aplica la regla LEGB (Local → Enclosing → Global → Built-in). No lo encuentra localmente, lo encuentra en el entorno enclosing (f1), que fue capturado en el momento en que wrapper fue creada. Ese entorno se llama celda de closure (cell object en CPython).

El patrón decorador. La función externa (uppercase_decorator) actúa como una fábrica que produce una función aumentada (wrapper) que envuelve a la original (greeting). Python ofrece el azúcar sintáctico @uppercase_decorator como atajo exacto para greeting = uppercase_decorator(greeting).

En código Python moderno siempre se agrega @functools.wraps(function) sobre wrapper para preservar el __name__, __doc__ y otros atributos de la función original, algo que este ejemplo básico aún no hace

Aplicando múltiples decoradores a una sola función.

```
# First Decorator
def uppercase_decorator(function):
    def wrapper():
        func = function()
        make_uppercase = func.upper()
        return make_uppercase
    return wrapper

# Second decorator
def split_string_decorator(function):
    def wrapper():
        func = function()
        splitted_string = func.split()
        return splitted_string
    return wrapper

#Decorators will be executed from bottom to top
@split_string_decorator
@uppercase_decorator    # order with decorators is important in this case - .upper() function
                        # does not work with lists
def greeting():
    return 'Welcome to Python'
print(greeting()) # ['WELCOME', 'TO', 'PYTHON']
```

Parte 1: Descripción línea por línea del código

```
python
# First Decorator
def uppercase_decorator(function):
    def wrapper():
        func = function()
        make_uppercase = func.upper()
        return make_uppercase
    return wrapper
```

Este primer decorador ya fue analizado en el ejercicio anterior. Recibe una función, define `wrapper` que llama a esa función, convierte el resultado a mayúsculas con `.upper()`, y retorna `wrapper` como objeto.

```
python
# Second decorator
def split_string_decorator(function):
```

Define el segundo decorador. Igual que el primero, recibe una función como parámetro —en este caso recibirá el `wrapper` que ya devolvió `uppercase_decorator`, no directamente `greeting`.

```
python
def wrapper():
```

Define la función interna envoltorio del segundo decorador. También se llama `wrapper` —son dos objetos distintos en el heap, con closures distintas, aunque compartan el mismo nombre de variable local.

```
python
    func = function()
```

Llama a la función recibida como argumento y guarda el resultado. Aquí `function` es el `wrapper` de `uppercase_decorator`, por lo que `func` recibirá la cadena "WELCOME TO PYTHON" (ya transformada a mayúsculas por el primer decorador).

```
python
    splitted_string = func.split()
```

Aplica el método `.split()` sobre el string recibido. Sin argumento, `split()` divide por espacios en blanco y retorna una lista: `['WELCOME', 'TO', 'PYTHON']`.

```
python
    return splitted_string
```

Retorna esa lista como resultado de ejecutar `wrapper`.

```
python
    return wrapper
```

Retorna el objeto función `wrapper` del segundo decorador sin ejecutarlo.

```
python
#Decorators will be executed from bottom to top
@split_string_decorator
@uppercase_decorator
def greeting():
    return 'Welcome to Python'
```

Aquí ocurre la composición. Python aplica los decoradores **de abajo hacia arriba**. Primero ejecuta `greeting = uppercase_decorator(greeting)`, con lo cual `greeting` pasa a ser `wrapper_de_uppercase`. Luego ejecuta `greeting = split_string_decorator(greeting)`, con lo cual `greeting` pasa a ser `wrapper_de_split`, cuyo `function` interno es `wrapper_de_uppercase`. El orden es crítico: si se invirtieran, `split` recibiría el string original en minúsculas, lo convertiría en lista

`['Welcome', 'to', 'Python']`, y luego `uppercase` intentaría llamar `.upper()` sobre una lista, lanzando `AttributeError`.

```
python
print(greeting()) # ['WELCOME', 'TO', 'PYTHON']
```

Llama a `wrapper_de_split()`, que a su vez llama a `wrapper_de_uppercase()`, que a su vez llama a `greeting()` original. El resultado fluye de vuelta como `['WELCOME', 'TO', 'PYTHON']` y `print` lo muestra.

Parte 2: Análisis detallado de cada imagen (30 pasos)

Steps 2–3: Definición del primer decorador en el Global Frame

Python ejecuta `def uppercase_decorator(function):` y crea en el heap el objeto función correspondiente. En el Global Frame aparece una sola entrada: `uppercase_decorator → function uppercase_decorator(function)`. Aún no existe `split_string_decorator` ni `greeting` como nombres en el entorno global.

The screenshot displays a Python 3.11 IDE with the following code:

```
1 # First Decorator
2 def uppercase_decorator(function):
3     def wrapper():
4         func = function()
5         make_uppercase = func.upper()
6         return make_uppercase
7     return wrapper
8
9 # Second decorator
10 def split_string_decorator(function):
11     def wrapper():
12         func = function()
13         splitted_string = func.split()
14         return splitted_string
15     return wrapper
16
17 #Decorators will be executed from bottom to top
18 @split_string_decorator
19 @uppercase_decorator    # order with decorators is important in this case
20 def greeting():
21     return 'Welcome to Python'
```

Below the code, a legend indicates:
→ line that just executed
→ next line to execute

At the bottom, navigation buttons are visible: `<< First`, `< Prev`, `Next >`, `Last >>`, and a page indicator `Step 2 of 30`.

On the right, a memory diagram titled "Print output (drag lower right corner to resize)" shows the state of the **Global frame**. It contains a single entry: `uppercase_decorator`. An arrow points from this entry to the **Objects** section, where the object `function uppercase_decorator(function)` is shown.

Steps 3–4: Se define `split_string_decorator` en el Global Frame

Python ejecuta `def split_string_decorator(function):` y agrega un segundo nombre al Global Frame. Ahora hay dos objetos función independientes en el heap. El visualizador muestra: `uppercase_decorator → function` `uppercase_decorator(function)` y `split_string_decorator → function` `split_string_decorator(function)`. Ninguna ha sido invocada todavía; sólo existen como objetos referenciados desde el Global Frame.

Python 3.11
[known limitations](#)

```
1 # First Decorator
2 def uppercase_decorator(function):
3     def wrapper():
4         func = function()
5         make_uppercase = func.upper()
6         return make_uppercase
7     return wrapper
8
9 # Second decorator
10 def split_string_decorator(function):
11     def wrapper():
12         func = function()
13         splitted_string = func.split()
14         return splitted_string
15     return wrapper
16
17 #Decorators will be executed from bottom to top
18 @split_string_decorator
19 @uppercase_decorator    # order with decorators is important in this case
20 def greeting():
21     return 'Welcome to Python'
```

[Edit this code](#)

→ line that just executed
→ next line to execute

<< First < Prev Next > Last >>

Step 3 of 30

Print output (drag lower right corner to resize)

Frames

Objects

Global frame

uppercase_decorator

split_string_decorator

function

uppercase_decorator(function)

function

split_string_decorator(function)

Python 3.11
[known limitations](#)

```
1 # First Decorator
2 def uppercase_decorator(function):
3     def wrapper():
4         func = function()
5         make_uppercase = func.upper()
6         return make_uppercase
7     return wrapper
8
9 # Second decorator
10 def split_string_decorator(function):
11     def wrapper():
12         func = function()
13         splitted_string = func.split()
14         return splitted_string
15     return wrapper
16
17 #Decorators will be executed from bottom to top
18 @split_string_decorator
19 @uppercase_decorator    # order with decorators is important in this case
20 def greeting():
21     return 'Welcome to Python'
```

[Edit this code](#)

→ line that just executed
→ next line to execute

<< First < Prev Next > Last >>

Step 4 of 30

Print output (drag lower right corner to resize)

Frames

Objects

Global frame

uppercase_decorator

split_string_decorator

function

uppercase_decorator(function)

function

split_string_decorator(function)

Steps 5–6: Python lee `@split_string_decorator` y `@uppercase_decorator`, define `greeting`

Este es el momento más importante del programa a nivel de carga. Python encuentra la pila de decoradores y la función `greeting`. Lo que ocurre internamente es la **desugaring** (transformación del azúcar sintáctico) en dos pasos secuenciales:

Paso A — Python ejecuta `def greeting(): return 'Welcome to Python'`, crea el objeto función `greeting()` en el heap, y lo enlaza temporalmente al nombre `greeting` en el Global Frame.

Paso B — Lee `@uppercase_decorator` (el decorador más cercano a `def`). Ejecuta `greeting = uppercase_decorator(greeting)`: abre el frame `f1` pasando el objeto `greeting()` como argumento `function`.

Python 3.11

[known limitations](#)

```

2 def uppercase_decorator(function):
3     def wrapper():
4         func = function()
5         make_uppercase = func.upper()
6         return make_uppercase
7     return wrapper
8
9 # Second decorator
10 def split_string_decorator(function):
11     def wrapper():
12         func = function()
13         splitted_string = func.split()
14         return splitted_string
15     return wrapper
16
17 #Decorators will be executed from bottom to top
18 @split_string_decorator
19 @uppercase_decorator    # order with decorators i
20 def greeting():
21     return 'Welcome to Python'
22 print(greeting())    # ['WELCOME', 'TO', 'PYTHON']

```

→ line that just executed

→ next line to execute

Step 5 of 30

Print output (drag lower right corner to resize)

Frames

Objects

Global frame

uppercase_decorator

split_string_decorator

function

uppercase_decorator(function)

function

split_string_decorator(function)

Python 3.11

[known limitations](#)

```

2 def uppercase_decorator(function):
3     def wrapper():
4         func = function()
5         make_uppercase = func.upper()
6         return make_uppercase
7     return wrapper
8
9 # Second decorator
10 def split_string_decorator(function):
11     def wrapper():
12         func = function()
13         splitted_string = func.split()
14         return splitted_string
15     return wrapper
16
17 #Decorators will be executed from bottom to top
18 @split_string_decorator
19 @uppercase_decorator    # order with decorators i
20 def greeting():
21     return 'Welcome to Python'
22 print(greeting())    # ['WELCOME', 'TO', 'PYTHON']

```

→ line that just executed

→ next line to execute

Step 6 of 30

Print output (drag lower right corner to resize)

Frames

Objects

Global frame

uppercase_decorator

split_string_decorator

function

uppercase_decorator(function)

function

split_string_decorator(function)

Steps 7–10: Se abre f1 para `uppercase_decorator`, se crea `wrapper[parent=f1]`

En el visualizador se ve por primera vez el frame `f1` debajo del Global Frame. Dentro de `f1` hay dos variables: `function` apuntando a `greeting()` en el heap, y `wrapper` apuntando a un nuevo objeto `wrapper()` [`parent=f1`]. La anotación `parent=f1` confirma la captura del closure: ese objeto función "recuerda" que su entorno léxico de definición fue `f1`.

La sentencia `return wrapper` activa el "Return value" en `f1`, señalando que `f1` está a punto de devolver el objeto `wrapper` al nivel que lo invocó. `f1` se destruirá, pero el objeto `wrapper()` [`parent=f1`] permanecerá en el heap porque será referenciado.

Python 3.11
[known limitations](#)

```
1 # First Decorator
2 def uppercase_decorator(function):
3     def wrapper():
4         func = function()
5         make_uppercase = func.upper()
6         return make_uppercase
7     return wrapper
8
9 # Second decorator
10 def split_string_decorator(function):
11     def wrapper():
12         func = function()
13         splitted_string = func.split()
14         return splitted_string
15     return wrapper
16
17 #Decorators will be executed from bottom to top
18 @split_string_decorator
19 @uppercase_decorator    # order with decorators is important in this case
20 def greeting():
21     return 'Welcome to Python!'
```

→ line that just executed

→ next line to execute

<< First

< Prev

Next >

Last >>

Step 7 of 30

Print output (drag lower right corner to resize)

Frames

Objects

Global frame

- uppercase_decorator
 - function
 - uppercase_decorator(function)
- split_string_decorator
 - function
 - split_string_decorator(function)

uppercase_decorator

- function
 - greeting()

Python 3.11
[known limitations](#)

```

1 # First Decorator
2 def uppercase_decorator(function):
3     def wrapper():
4         func = function()
5         make_uppercase = func.upper()
6         return make_uppercase
7     return wrapper
8
9 # Second decorator
10 def split_string_decorator(function):
11     def wrapper():
12         func = function()
13         splitted_string = func.split()
14         return splitted_string
15     return wrapper
16
17 #Decorators will be executed from bottom to top
18 @split_string_decorator
19 @uppercase_decorator    # order with decorators is important in this case
20 def greeting():
21     return 'Welcome to Python!'

```

[Edit this code](#)

→ line that just executed
→ next line to execute

Step 9 of 30

Print output (drag lower right corner to resize)

Frames

Objects

Python 3.11
[known limitations](#)

```

1 # First Decorator
2 def uppercase_decorator(function):
3     def wrapper():
4         func = function()
5         make_uppercase = func.upper()
6         return make_uppercase
7     return wrapper
8
9 # Second decorator
10 def split_string_decorator(function):
11     def wrapper():
12         func = function()
13         splitted_string = func.split()
14         return splitted_string
15     return wrapper
16
17 #Decorators will be executed from bottom to top
18 @split_string_decorator
19 @uppercase_decorator    # order with decorators is important in this case
20 def greeting():
21     return 'Welcome to Python!'

```

[Edit this code](#)

→ line that just executed
→ next line to execute

Step 10 of 30

Print output (drag lower right corner to resize)

Frames

Objects

Step 11: f1 retorna, **greeting** ahora apunta a **wrapper** [parent=f1]

f1 desaparece de la pila. El nombre **greeting** en el Global Frame ya no apunta a **greeting()** original, sino a **wrapper()** [parent=f1]. La función original sigue viva en el heap porque el closure de **wrapper** la referencia a través de **function**.

Python 3.11

[known limitations](#)

```

1 # First Decorator
2 def uppercase_decorator(function):
3     def wrapper():
4         func = function()
5         make_uppercase = func.upper()
6         return make_uppercase
7     return wrapper
8
9 # Second decorator
10 def split_string_decorator(function):
11     def wrapper():
12         func = function()
13         splitted_string = func.split()
14         return splitted_string
15     return wrapper
16
17 #Decorators will be executed from bottom to top
18 @split_string_decorator
19 @uppercase_decorator    # order with decorators i
20 def greeting():
21     return 'Welcome to Python!'

```

[Edit this code](#)

line that just executed

next line to execute

<< First

< Prev

Next >

Last >>

Step 11 of 30

Print output (drag lower right corner to resize)

Frames

Objects

Global frame

uppercase_decorator

split_string_decorator

function

uppercase_decorator(function)

function

split_string_decorator(function)

f1: uppercase_decorator

function

greeting()

wrapper

wrapper() [parent=f1]

Return value

Steps 12–13: Se procesa @split_string_decorator

Python ahora ejecuta `greeting = split_string_decorator(greeting)`. El argumento que pasa es el `wrapper() [parent=f1]` que acaba de quedar como valor de `greeting`. Se abre un nuevo frame, el frame `split_string_decorator` (que el visualizador llama f2 más adelante). En este frame `function` apunta a `wrapper() [parent=f1]`.

Python 3.11
[known limitations](#)

```
1 # First Decorator
2 def uppercase_decorator(function):
3     def wrapper():
4         func = function()
5         make_uppercase = func.upper()
6         return make_uppercase
7     return wrapper
8
9 # Second decorator
10 def split_string_decorator(function):
11     def wrapper():
12         func = function()
13         splitted_string = func.split()
14         return splitted_string
15     return wrapper
16
17 #Decorators will be executed from bottom to top
18 @split_string_decorator
19 @uppercase_decorator    # order with decorators is important in this case
20 def greeting():
21     return 'Welcome to Python'
```

Print output (drag lower right corner to resize)

Frames

Objects

Global frame
uppercase_decorator
split_string_decorator

f1: uppercase_decorator
function
wrapper
Return value

split_string_decorator
function

function
uppercase_decorator(function)
function
split_string_decorator(function)
function
greeting()
function
wrapper() [parent=f1]

→ line that just executed

→ next line to execute

<< First

< Prev

Next >

Last >>

Step 12 of 30

Python 3.11
[known limitations](#)

```
1 # First Decorator
2 def uppercase_decorator(function):
3     def wrapper():
4         func = function()
5         make_uppercase = func.upper()
6         return make_uppercase
7     return wrapper
8
9 # Second decorator
10 def split_string_decorator(function):
11     def wrapper():
12         func = function()
13         splitted_string = func.split()
14         return splitted_string
15     return wrapper
16
17 #Decorators will be executed from bottom to top
18 @split_string_decorator
19 @uppercase_decorator    # order with decorators is important in this case
20 def greeting():
21     return 'Welcome to Python'
```

Print output (drag lower right corner to resize)

Frames

Objects

Global frame
uppercase_decorator
split_string_decorator

f1: uppercase_decorator
function
wrapper
Return value

split_string_decorator
function

function
uppercase_decorator(function)
function
split_string_decorator(function)
function
greeting()
function
wrapper() [parent=f1]

→ line that just executed

→ next line to execute

<< First

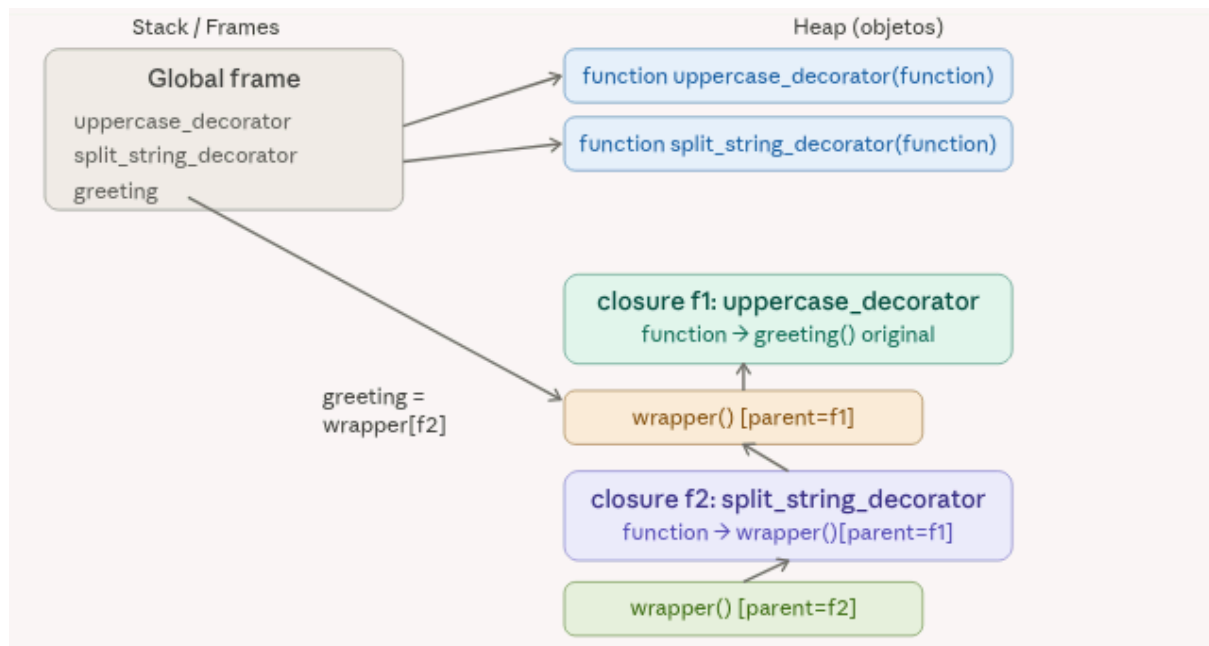
< Prev

Next >

Last >>

Step 13 of 30

Steps 14–15: Se crea **wrapper[parent=f2]** dentro del frame de **split_string_decorator**



El visualizador muestra simultáneamente f1 (aún visible en el heap porque sus objetos siguen siendo referenciados) y el nuevo frame de `split_string_decorator`. Dentro de este frame se ejecuta `def wrapper():`, creando un segundo objeto `wrapper() [parent=f2]` —distinto del primero, con su propio closure que captura `function = wrapper[parent=f1]`. La sentencia `return wrapper` marca el Return value del frame f2.

Python 3.11
[known limitations](#)

```

1 # First Decorator
2 def uppercase_decorator(function):
3     def wrapper():
4         func = function()
5         make_uppercase = func.upper()
6         return make_uppercase
7     return wrapper
8
9 # Second decorator
10 def split_string_decorator(function):
11     def wrapper():
12         func = function()
13         splitted_string = func.split()
14         return splitted_string
15     return wrapper
16
17 #Decorators will be executed from bottom to top
18 @split_string_decorator
19 @uppercase_decorator    # order with decorators is important in this case
20 def greeting():
21     return 'Welcome to Python!'

```

→ line that just executed

→ next line to execute

<< First

< Prev

Next >

Last >>

Step 14 of 30

Print output (drag lower right corner to resize)

Frames

Objects

Global frame

uppercase_decorator

split_string_decorator

f1: uppercase_decorator

function

wrapper

Return value

f2: split_string_decorator

function

wrapper

function uppercase_decorator(function)

function split_string_decorator(function)

function greeting()

function wrapper() [parent=f1]

function wrapper() [parent=f2]


```

Python 3.11
known limitations
1 # First Decorator
2 def uppercase_decorator(function):
3     def wrapper():
4         func = function()
5         make_uppercase = func.upper()
6         return make_uppercase
7     return wrapper
8
9 # Second decorator
10 def split_string_decorator(function):
11     def wrapper():
12         func = function()
13         splitted_string = func.split()
14         return splitted_string
15     return wrapper
16
17 @split_string_decorator
18 @uppercase_decorator # order with decorators i
19 def greeting():
20     return 'Welcome to Python'
21 print(greeting()) # ['WELCOME', 'TO', 'PYTHON']

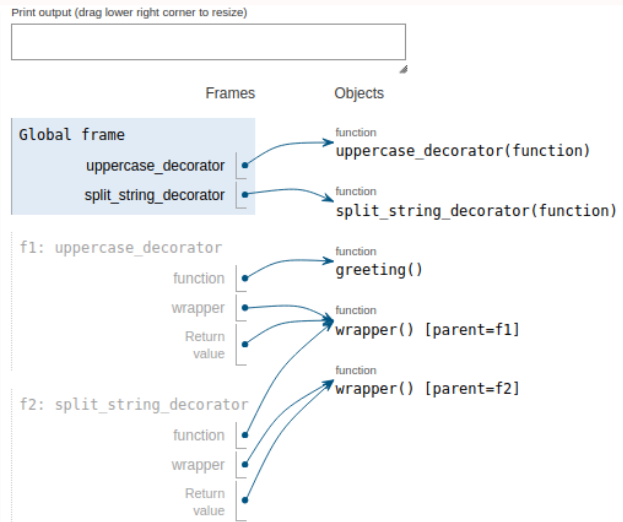
```

[Edit this code](#)

→ line that just executed
 → next line to execute

<< First < Prev Next > Last >>

Step 16 of 30



```

Python 3.11
known limitations
1 # First Decorator
2 def uppercase_decorator(function):
3     def wrapper():
4         func = function()
5         make_uppercase = func.upper()
6         return make_uppercase
7     return wrapper
8
9 # Second decorator
10 def split_string_decorator(function):
11     def wrapper():
12         func = function()
13         splitted_string = func.split()
14         return splitted_string
15     return wrapper
16
17 @split_string_decorator
18 @uppercase_decorator # order with decorators i
19 def greeting():
20     return 'Welcome to Python'
21 print(greeting()) # ['WELCOME', 'TO', 'PYTHON']

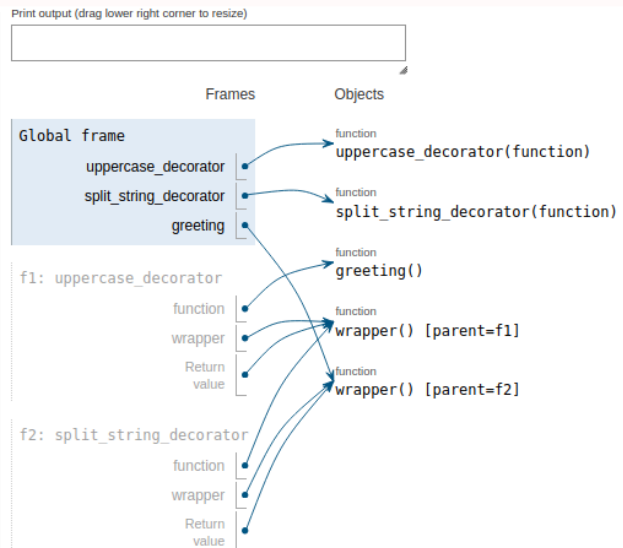
```

[Edit this code](#)

→ line that just executed
 → next line to execute

<< First < Prev Next > Last >>

Step 17 of 30



Steps 18–19: `print(greeting())` activa la cadena completa

Se ejecuta `greeting()`. Python busca el nombre `greeting` en el Global Frame, encuentra `wrapper[parent=f2]`, y lo llama. Se abre el frame de `wrapper[parent=f2]` (frame de `split_string_decorator`).

Python 3.11
[known limitations](#)

```

1 # First decorator
2 def uppercase_decorator(function):
3     def wrapper():
4         func = function()
5         make_uppercase = func.upper()
6         return make_uppercase
7     return wrapper
8
9 # Second decorator
10 def split_string_decorator(function):
11     def wrapper():
12         func = function()
13         splitted_string = func.split()
14         return splitted_string
15     return wrapper
16
17 @split_string_decorator
18 @uppercase_decorator    # order with decorators is important in this case
19 def greeting():
20     return 'Welcome to Python'
21 print(greeting())    # ['WELCOME', 'TO', 'PYTHON']

```

[Edit this code](#)

→ line that just executed
→ next line to execute

Step 18 of 30

Print output (drag lower right corner to resize)

Frames

Global frame

- uppercase_decorator
- split_string_decorator
- greeting

f1: uppercase_decorator

- function
- wrapper
- Return value

f2: split_string_decorator

- function
- wrapper
- Return value

Objects

- function uppercase_decorator(function)
- function split_string_decorator(function)
- function greeting()
- function wrapper() [parent=f1]
- function wrapper() [parent=f2]

wrapper [parent=f2]

Python 3.11
[known limitations](#)

```

1 # First decorator
2 def uppercase_decorator(function):
3     def wrapper():
4         func = function()
5         make_uppercase = func.upper()
6         return make_uppercase
7     return wrapper
8
9 # Second decorator
10 def split_string_decorator(function):
11     def wrapper():
12         func = function()
13         splitted_string = func.split()
14         return splitted_string
15     return wrapper
16
17 @split_string_decorator
18 @uppercase_decorator    # order with decorators is important in this case
19 def greeting():
20     return 'Welcome to Python'
21 print(greeting())    # ['WELCOME', 'TO', 'PYTHON']

```

[Edit this code](#)

→ line that just executed
→ next line to execute

Step 19 of 30

Print output (drag lower right corner to resize)

Frames

Global frame

- uppercase_decorator
- split_string_decorator
- greeting

f1: uppercase_decorator

- function
- wrapper
- Return value

f2: split_string_decorator

- function
- wrapper
- Return value

Objects

- function uppercase_decorator(function)
- function split_string_decorator(function)
- function greeting()
- function wrapper() [parent=f1]
- function wrapper() [parent=f2]

wrapper [parent=f2]

Steps 20–21: Dentro de `wrapper [parent=f2]`, se ejecuta `func = function()`

`function` en este closure es `wrapper [parent=f1]`. Al llamarlo, se abre otro frame debajo en la pila: el frame de `wrapper [parent=f1]` (frame de `uppercase_decorator`).

En el visualizador se ven simultáneamente **dos frames activos** llamados **wrapper** —uno con **parent=f2** y otro con **parent=f1**— lo cual ilustra perfectamente el concepto de pila de llamadas (call stack) con closures anidados.

Python 3.11 [known limitations](#)

```

1 # First decorator
2 def uppercase_decorator(function):
3     def wrapper():
4         func = function()
5         make_uppercase = func.upper()
6         return make_uppercase
7     return wrapper
8
9 # Second decorator
10 def split_string_decorator(function):
11     def wrapper():
12         func = function()
13         splitted_string = func.split()
14         return splitted_string
15     return wrapper
16
17 @split_string_decorator
18 @uppercase_decorator    # order with decorators is important in this case
19 def greeting():
20     return 'Welcome to Python'
21 print(greeting())    # ['WELCOME', 'TO', 'PYTHON']

```

[Edit this code](#)

→ line that just executed
→ next line to execute

Step 20 of 30

Print output (drag lower right corner to resize)

Frames

Global frame

- uppercase_decorator
- split_string_decorator
- greeting

f1: uppercase_decorator

- function
- wrapper
- Return value

f2: split_string_decorator

- function
- wrapper
- Return value

Objects

- function uppercase_decorator(function)
- function split_string_decorator(function)
- function greeting()
- function wrapper() [parent=f1]
- function wrapper() [parent=f2]

wrapper [parent=f2]

wrapper [parent=f1]

Python 3.11 [known limitations](#)

```

1 # First decorator
2 def uppercase_decorator(function):
3     def wrapper():
4         func = function()
5         make_uppercase = func.upper()
6         return make_uppercase
7     return wrapper
8
9 # Second decorator
10 def split_string_decorator(function):
11     def wrapper():
12         func = function()
13         splitted_string = func.split()
14         return splitted_string
15     return wrapper
16
17 @split_string_decorator
18 @uppercase_decorator    # order with decorators is important in this case
19 def greeting():
20     return 'Welcome to Python'
21 print(greeting())    # ['WELCOME', 'TO', 'PYTHON']

```

[Edit this code](#)

→ line that just executed
→ next line to execute

Step 21 of 30

Print output (drag lower right corner to resize)

Frames

Global frame

- uppercase_decorator
- split_string_decorator
- greeting

f1: uppercase_decorator

- function
- wrapper
- Return value

f2: split_string_decorator

- function
- wrapper
- Return value

Objects

- function uppercase_decorator(function)
- function split_string_decorator(function)
- function greeting()
- function wrapper() [parent=f1]
- function wrapper() [parent=f2]

wrapper [parent=f2]

wrapper [parent=f1]

Steps 22–24: `wrapper[parent=f1]` llama a `greeting()` original

Dentro del segundo frame, `function()` apunta ahora al `greeting()` original (capturado cuando se aplicó `@uppercase_decorator`). Se abre el frame más profundo de la pila: `greeting`. En el visualizador se puede ver la pila completa con tres niveles activos: Global Frame → f2 (split) → f1 (upper) → greeting. El frame de `greeting` retorna "Welcome to Python".

Python 3.11 [known limitations](#)

```

1 # First decorator
2 def uppercase_decorator(function):
3     def wrapper():
4         func = function()
5         make_uppercase = func.upper()
6         return make_uppercase
7     return wrapper
8
9 # Second decorator
10 def split_string_decorator(function):
11     def wrapper():
12         func = function()
13         splitted_string = func.split()
14         return splitted_string
15     return wrapper
16
17 @split_string_decorator
18 @uppercase_decorator    # order with decorators is important in this case
19 def greeting():
20     return 'Welcome to Python'
21 print(greeting())    # ['WELCOME', 'TO', 'PYTHON']

```

[Edit this code](#)

→ line that just executed
→ next line to execute

Step 22 of 30

Print output (drag lower right corner to resize)

Frames	Objects
Global frame	uppercase_decorator split_string_decorator greeting
f1: uppercase_decorator	function wrapper Return value
f2: split_string_decorator	function wrapper Return value

wrapper [parent=f2]
wrapper [parent=f1]
greeting

Python 3.11 [known limitations](#)

```

1 # First decorator
2 def uppercase_decorator(function):
3     def wrapper():
4         func = function()
5         make_uppercase = func.upper()
6         return make_uppercase
7     return wrapper
8
9 # Second decorator
10 def split_string_decorator(function):
11     def wrapper():
12         func = function()
13         splitted_string = func.split()
14         return splitted_string
15     return wrapper
16
17 @split_string_decorator
18 @uppercase_decorator    # order with decorators is important in this case
19 def greeting():
20     return 'Welcome to Python'
21 print(greeting())    # ['WELCOME', 'TO', 'PYTHON']

```

[Edit this code](#)

→ line that just executed
→ next line to execute

Step 23 of 30

Print output (drag lower right corner to resize)

Frames	Objects
Global frame	uppercase_decorator split_string_decorator greeting
f1: uppercase_decorator	function wrapper Return value
f2: split_string_decorator	function wrapper Return value

wrapper [parent=f2]
wrapper [parent=f1]
greeting

```

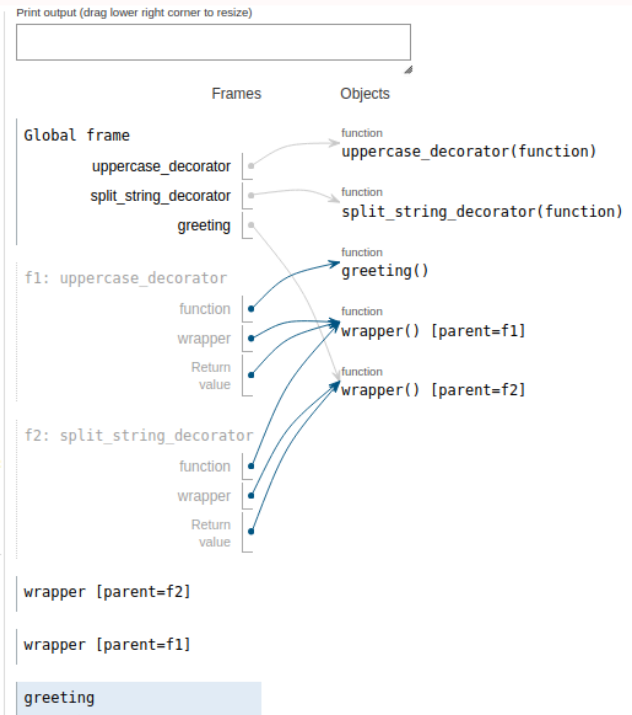
Python 3.11
known limitations
1 # First decorator
2 def uppercase_decorator(function):
3     def wrapper():
4         func = function()
5         make_uppercase = func.upper()
6         return make_uppercase
7     return wrapper
8
9 # Second decorator
10 def split_string_decorator(function):
11     def wrapper():
12         func = function()
13         splitted_string = func.split()
14         return splitted_string
15     return wrapper
16
17 @split_string_decorator
18 @uppercase_decorator # order with decorators is important in this case
19 def greeting():
20     return 'Welcome to Python'
21 print(greeting()) # ['WELCOME', 'TO', 'PYTHON']

```

→ line that just executed
 → next line to execute

<< First < Prev Next > Last >>

Step 24 of 30



Steps 25–27: `wrapper[parent=f1]` transforma el string

Con el retorno de `greeting()`, el frame `f1` continúa: `func = "Welcome to Python"`. Luego ejecuta `make_uppercase = func.upper()` produciendo `"WELCOME TO PYTHON"`. El visualizador muestra en el frame de `wrapper[parent=f1]`: `func = "Welcome to Python"` y `make_uppercase = "WELCOME TO PYTHON"`. Finalmente `return make_uppercase` activa el Return value de este frame.

```

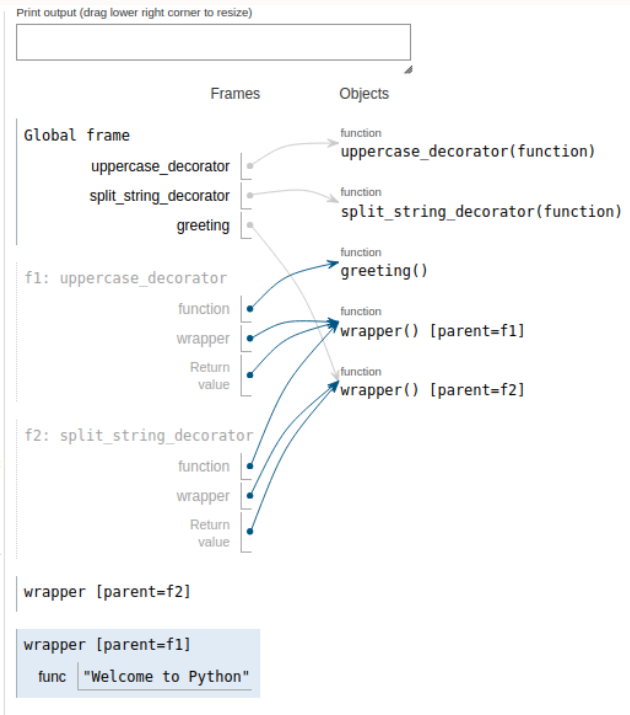
Python 3.11
known limitations
1 # First decorator
2 def uppercase_decorator(function):
3     def wrapper():
4         func = function()
5         make_uppercase = func.upper()
6         return make_uppercase
7     return wrapper
8
9 # Second decorator
10 def split_string_decorator(function):
11     def wrapper():
12         func = function()
13         splitted_string = func.split()
14         return splitted_string
15     return wrapper
16
17 @split_string_decorator
18 @uppercase_decorator # order with decorators is important in this case
19 def greeting():
20     return 'Welcome to Python'
21 print(greeting()) # ['WELCOME', 'TO', 'PYTHON']

```

[Edit this code](#)

line that just executed
 next line to execute

Step 25 of 30



```

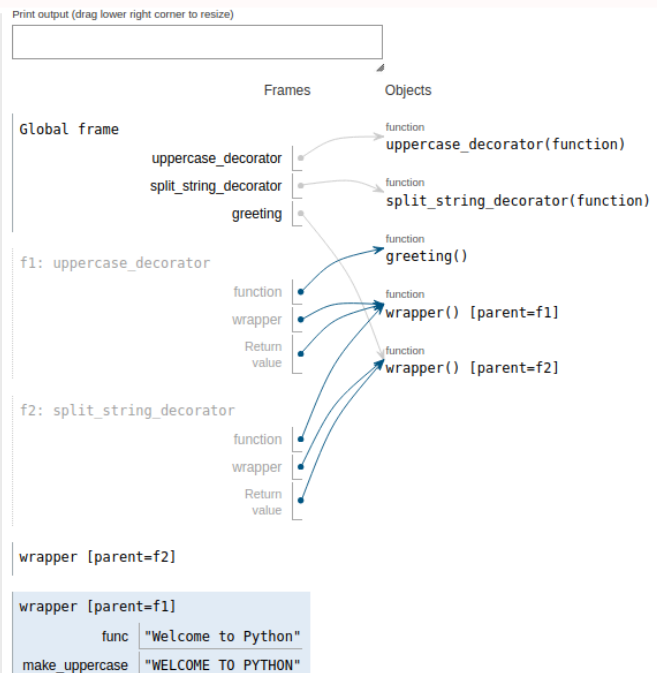
Python 3.11
known limitations
1 # First decorator
2 def uppercase_decorator(function):
3     def wrapper():
4         func = function()
5         make_uppercase = func.upper()
6         return make_uppercase
7     return wrapper
8
9 # Second decorator
10 def split_string_decorator(function):
11     def wrapper():
12         func = function()
13         splitted_string = func.split()
14         return splitted_string
15     return wrapper
16
17 @split_string_decorator
18 @uppercase_decorator # order with decorators is important in this case
19 def greeting():
20     return 'Welcome to Python'
21 print(greeting()) # ['WELCOME', 'TO', 'PYTHON']

```

[Edit this code](#)

line that just executed
 next line to execute

Step 26 of 30



Python 3.11

known limitations

```

1 # first decorator
2 def uppercase_decorator(function):
3     def wrapper():
4         func = function()
5         make_uppercase = func.upper()
6         return make_uppercase
7     return wrapper
8
9 # Second decorator
10 def split_string_decorator(function):
11     def wrapper():
12         func = function()
13         splitted_string = func.split()
14         return splitted_string
15     return wrapper
16
17 @split_string_decorator
18 @uppercase_decorator # order with decorators i
19 def greeting():
20     return 'Welcome to Python'
21 print(greeting()) # ['WELCOME', 'TO', 'PYTHON']

```

Edit this code

→ line that just executed

→ next line to execute

<< First

< Prev

Next >

Last >>

Step 27 of 30

Print output (drag lower right corner to resize)

Frames

Objects

Global frame

uppercase_decorator

split_string_decorator

greeting

f1: uppercase_decorator

function

wrapper

Return value

f2: split_string_decorator

function

wrapper

Return value

wrapper [parent=f2]

wrapper [parent=f1]

func

make_uppercase

function uppercase_decorator(function)

function split_string_decorator(function)

function greeting()

function wrapper() [parent=f1]

function wrapper() [parent=f2]

func "Welcome to Python"

make_uppercase "WELCOME TO PYTHON"

Steps 28–29: `wrapper[parent=f2]` recibe el string mayúscula y aplica `.split()`

El frame f2 continúa con `func = "WELCOME TO PYTHON"`. Ejecuta `splitted_string = func.split()`. El visualizador muestra el objeto lista en el heap como tres celdas indexadas: `[0] "WELCOME"`, `[1] "TO"`, `[2] "PYTHON"`. Esta es la representación interna de una lista en CPython: un array de punteros a objetos. El frame f2 marca su Return value apuntando a esa lista.

```

Python 3.11
known limitations
1 # First decorator
2 def uppercase_decorator(function):
3     def wrapper():
4         func = function()
5         make_uppercase = func.upper()
6         return make_uppercase
7     return wrapper
8
9 # Second decorator
10 def split_string_decorator(function):
11     def wrapper():
12         func = function()
13         splitted_string = func.split()
14         return splitted_string
15     return wrapper
16
17 @split_string_decorator
18 @uppercase_decorator    # order with decorators is important in this example
19 def greeting():
20     return 'Welcome to Python'
21 print(greeting())    # ['WELCOME', 'TO', 'PYTHON']

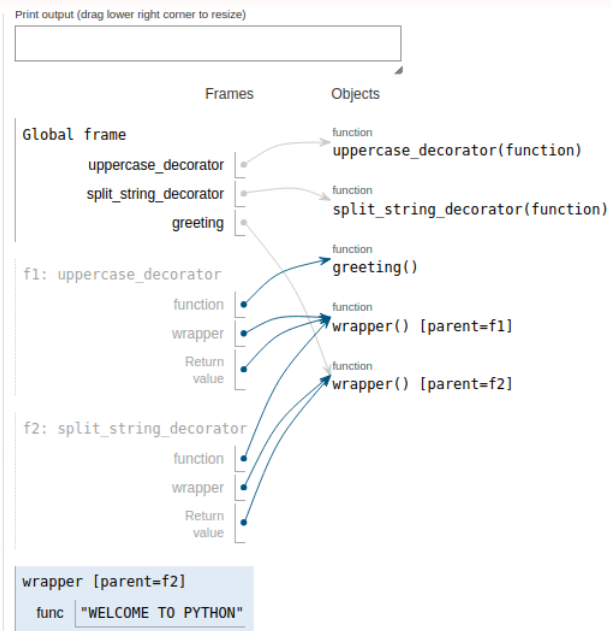
```

[Edit this code](#)

→ line that just executed
→ next line to execute

<< First < Prev Next > Last >>

Step 28 of 30



```

Python 3.11
known limitations
1 # First decorator
2 def uppercase_decorator(function):
3     def wrapper():
4         func = function()
5         make_uppercase = func.upper()
6         return make_uppercase
7     return wrapper
8
9 # Second decorator
10 def split_string_decorator(function):
11     def wrapper():
12         func = function()
13         splitted_string = func.split()
14         return splitted_string
15     return wrapper
16
17 @split_string_decorator
18 @uppercase_decorator    # order with decorators is important in this example
19 def greeting():
20     return 'Welcome to Python'
21 print(greeting())    # ['WELCOME', 'TO', 'PYTHON']

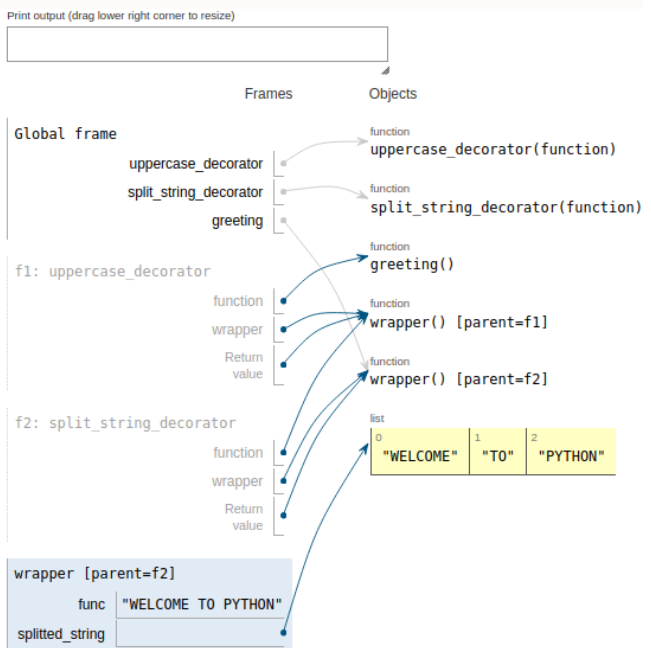
```

[Edit this code](#)

→ line that just executed
→ next line to execute

<< First < Prev Next > Last >>

Step 29 of 30



Step 30 (final): `print()` escribe el resultado, todos los frames se limpian

El frame f2 retorna la lista. `print()` la recibe, la serializa como `['WELCOME', 'TO', 'PYTHON']` y la escribe en stdout —visible en la franja "Print output" del visualizador. Todos los frames de activación se destruyen. En el heap solo sobreviven los objetos que siguen referenciados desde el Global Frame.

Python 3.11
[known limitations](#)

```

1 # first decorator
2 def uppercase_decorator(function):
3     def wrapper():
4         func = function()
5         make_uppercase = func.upper()
6         return make_uppercase
7     return wrapper
8
9 # Second decorator
10 def split_string_decorator(function):
11     def wrapper():
12         func = function()
13         splitted_string = func.split()
14         return splitted_string
15     return wrapper
16
17 @split_string_decorator
18 @uppercase_decorator    # order with decorators is important in this case
19 def greeting():
20     return 'Welcome to Python'
21 print(greeting())    # ['WELCOME', 'TO', 'PYTHON']

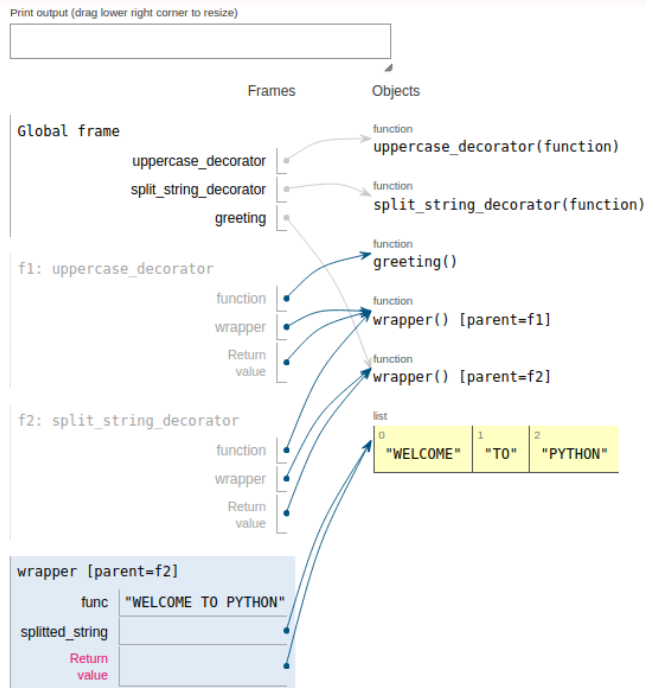
```

[Edit this code](#)

→ line that just executed
→ next line to execute

<< First < Prev Next > Last >>

Step 30 of 30



Python 3.11
[known limitations](#)

```

1 # first decorator
2 def uppercase_decorator(function):
3     def wrapper():
4         func = function()
5         make_uppercase = func.upper()
6         return make_uppercase
7     return wrapper
8
9 # Second decorator
10 def split_string_decorator(function):
11     def wrapper():
12         func = function()
13         splitted_string = func.split()
14         return splitted_string
15     return wrapper
16
17 @split_string_decorator
18 @uppercase_decorator    # order with decorators is important in this case
19 def greeting():
20     return 'Welcome to Python'
21 print(greeting())    # ['WELCOME', 'TO', 'PYTHON']

```

[Edit this code](#)

→ line that just executed
→ next line to execute

<< First < Prev Next > Last >>

Done running (30 steps)

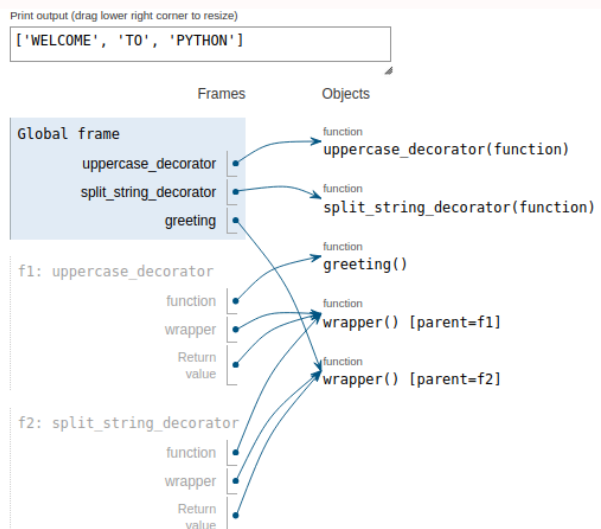
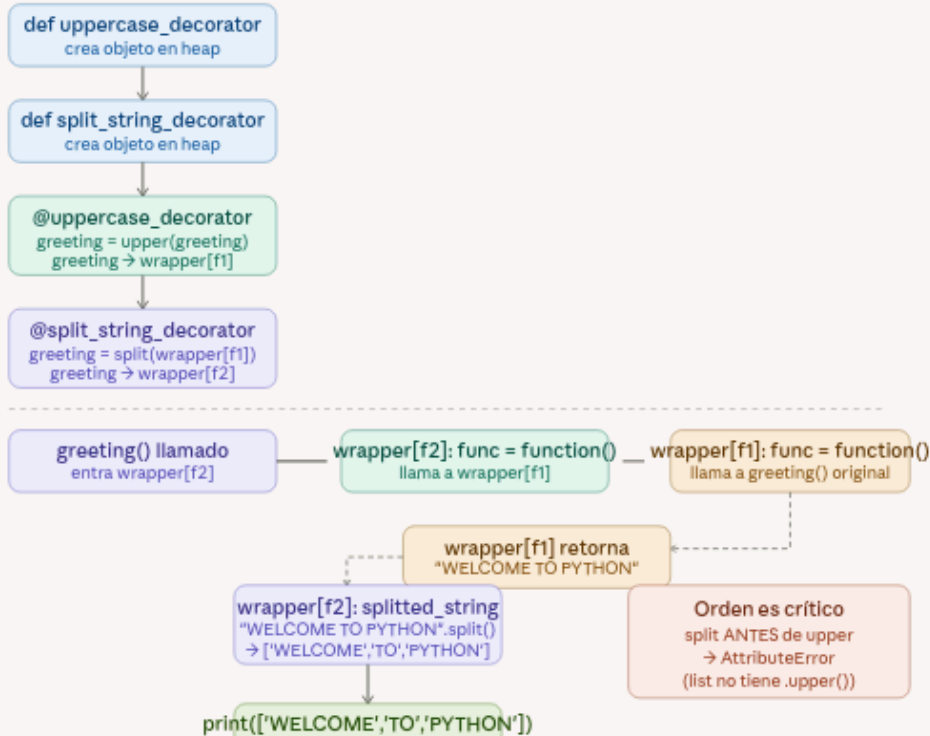


Diagrama completo de la cadena de ejecución

Fase 1: Definición (steps 2–17)



Síntesis conceptual avanzada

Este ejercicio de 30 pasos revela tres mecanismos de CPython operando en conjunto que no son evidentes en la superficie del código:

Composición de funciones mediante reasignación de nombres. El operador `@` no es magia —es azúcar sintáctico que reasigna el nombre de la función decorada. Después de procesar ambos decoradores, `greeting` en el Global Frame ya no apunta al objeto función original, sino a dos capas de `wrapper` anidadas. La función original `greeting()` queda "enterrada" bajo dos closures.

Dos frames `wrapper` activos simultáneamente en la call stack. En los steps 20–22 el visualizador muestra algo que sorprende a quienes aprenden Python: dos frames con el mismo nombre `wrapper` apilados. Esto ocurre porque son dos objetos función distintos en el heap con el mismo nombre de variable local, cada uno con su propio entorno de ejecución aislado. El intérprete los distingue por su dirección de memoria (su identidad de objeto), no por su nombre.

Transformación del tipo de datos a través de la cadena. La cadena entera produce una conversión de tipos: `str` → `str` (mayúsculas) → `list`. Esto ilustra un principio de diseño clave sobre los decoradores: cada decorador en la cadena debe ser consciente del tipo que recibe. Si el orden se invirtiera —`@uppercase_decorator` encima de `@split_string_decorator`— se obtendría un `AttributeError` porque `list` no tiene

método `.upper()`. En producción, `functools.wraps` y anotaciones de tipo (`typing.Callable`) ayudan a documentar y hacer cumplir estas restricciones de tipo en las cadenas de decoradores.

```
def decorator_with_parameters(function):
    def wrapper_accepting_parameters(para1, para2, para3):
        function(para1, para2, para3)
        print("I live in {}".format(para3))
    return wrapper_accepting_parameters
```

```
@decorator_with_parameters
def print_full_name(first_name, last_name, country):
    print("I am {} {}. I love to teach.".format(
        first_name, last_name))
```

```
print_full_name("Asabeneh", "Yetayeh", 'Finland')
```

Parte 1: Descripción línea por línea

```
def decorator_with_parameters(function):
```

Define la función decoradora. Como siempre, recibe otra función como argumento. El nombre es descriptivo: este decorador está diseñado específicamente para envolver funciones que aceptan parámetros.

```
python
```

```
    def wrapper_accepting_parameters(para1, para2, para3):
```

Define la función interna envoltorio. A diferencia de los ejemplos anteriores donde `wrapper()` no tenía parámetros, aquí `wrapper_accepting_parameters` declara tres parámetros explícitos. Estos son los parámetros que recibirá cuando se llame a la función decorada. Es fundamental que la firma del `wrapper` coincida con la firma de la función que va a envolver —de lo contrario Python lanzará `TypeError` al momento de la llamada.

```
python
```

```
        function(para1, para2, para3)
```

Llama a la función original pasándole los tres parámetros que recibió el wrapper. Esto ejecuta `print_full_name` con los argumentos reales, imprimiendo la primera línea de salida: `"I am Asabeneh Yetayeh. I love to teach."`. Nótese que no se guarda el valor de retorno —en este ejemplo la función original no retorna nada relevante (retorna `None`).

```
python
```

```
print("I live in {}".format(para3))
```

Esta es la funcionalidad **añadida** por el decorador. Usa `para3` (el país) directamente porque `wrapper_accepting_parameters` tiene acceso a él como parámetro local. Imprime la segunda línea: "I live in Finland".

```
python
return wrapper_accepting_parameters
```

Retorna el objeto función `wrapper_accepting_parameters` sin ejecutarlo. Desde este momento, el nombre `print_full_name` en el Global Frame apuntará a este wrapper.

```
python
@decorator_with_parameters
def print_full_name(first_name, last_name, country):
    print("I am {} {}. I love to teach.".format(
        first_name, last_name))
```

Python ejecuta el `def`, crea el objeto función `print_full_name`, y luego aplica el decorador: `print_full_name = decorator_with_parameters(print_full_name)`. El resultado es que `print_full_name` queda enlazado a `wrapper_accepting_parameters`, que tiene en su closure la referencia a la función original.

```
python
print_full_name("Asabeneh", "Yetayeh", 'Finland')
```

Llama a `wrapper_accepting_parameters("Asabeneh", "Yetayeh", "Finland")`. Internamente llama a la función original con esos mismos argumentos y luego ejecuta la lógica adicional del decorador.

Parte 2: Análisis de la ejecución a nivel avanzado

El punto conceptualmente nuevo respecto a los ejercicios anteriores es la **propagación de argumentos a través del wrapper**. A continuación se ilustra cada fase clave.

Fase 1 — Definición y decoración

Python crea `decorator_with_parameters` en el heap, luego crea `print_full_name` original, aplica el decorador abriendo un frame donde `function = print_full_name_original`, crea `wrapper_accepting_parameters [parent=f1]` con ese closure, retorna el wrapper, y reasigna `print_full_name` al wrapper.

Steps 2–3: Solo `decorator_with_parameters` existe en el Global Frame

Python ejecuta `def decorator_with_parameters(function):` y crea el objeto función en el heap. El Global Frame tiene una única entrada. `print_full_name` todavía no existe como nombre. El intérprete aún no llegó al bloque `@decorator_with_parameters`.

The screenshot shows a Python 3.11 IDE with the following code:

```
1 def decorator_with_parameters(function):
2     def wrapper_accepting_parameters(para1, para2,
3       function(para1, para2, para3)
4       print("I live in {}".format(para3))
5     return wrapper_accepting_parameters
6
7 @decorator_with_parameters
8 def print_full_name(first_name, last_name, country
9   print("I am {} {}. I love to teach.".format(
10     first_name, last_name))
11
12 print_full_name("Asabeneh", "Yetayeh", 'Finland')
```

Below the code, there is a legend: a green arrow for "line that just executed" and a red arrow for "next line to execute". The red arrow points to line 7. Navigation buttons include "<< First", "< Prev", "Next >", and "Last >>". The status bar indicates "Step 2 of 19".

On the right, a "Print output" box is empty. Below it, a diagram titled "Frames" and "Objects" shows the "Global frame" containing the entry "decorator_with_parameters". An arrow labeled "function" points from this entry to the object "decorator_with_parameters(function)" in the "Objects" section.

Steps 3–4: Python llega a `@decorator_with_parameters`

El visualizador muestra la flecha de "next line to execute" apuntando a la línea `@decorator_with_parameters` (línea 7) y luego a `def print_full_name` (línea 8). Python procesa ambas juntas como una unidad: primero ejecuta `def print_full_name(...)` para crear el objeto función en el heap, y luego aplica el decorador. El Global Frame aún solo muestra `decorator_with_parameters` porque la reasignación aún no ocurrió.

Python 3.11

[known limitations](#)

```

1 def decorator_with_parameters(function):
2     def wrapper_accepting_parameters(para1, para2,
3         function(para1, para2, para3)
4         print("I live in {}".format(para3))
5     return wrapper_accepting_parameters
6
7 @decorator_with_parameters
8 def print_full_name(first_name, last_name, country)
9     print("I am {} {}. I love to teach.".format(
10         first_name, last_name))
11
12 print_full_name("Asabeneh", "Yetayeh",'Finland')

```

line that just executed

next line to execute

<< First

< Prev

Next >

Last >>

Step 3 of 19

Print output (drag lower right corner to resize)

Frames

Objects

Global frame

decorator_with_parameters

function

decorator_with_parameters(function)

Python 3.11

[known limitations](#)

```

1 def decorator_with_parameters(function):
2     def wrapper_accepting_parameters(para1, para2,
3         function(para1, para2, para3)
4         print("I live in {}".format(para3))
5     return wrapper_accepting_parameters
6
7 @decorator_with_parameters
8 def print_full_name(first_name, last_name, country)
9     print("I am {} {}. I love to teach.".format(
10         first_name, last_name))
11
12 print_full_name("Asabeneh", "Yetayeh",'Finland')

```

line that just executed

next line to execute

<< First

< Prev

Next >

Last >>

Step 4 of 19

Print output (drag lower right corner to resize)

Frames

Objects

Global frame

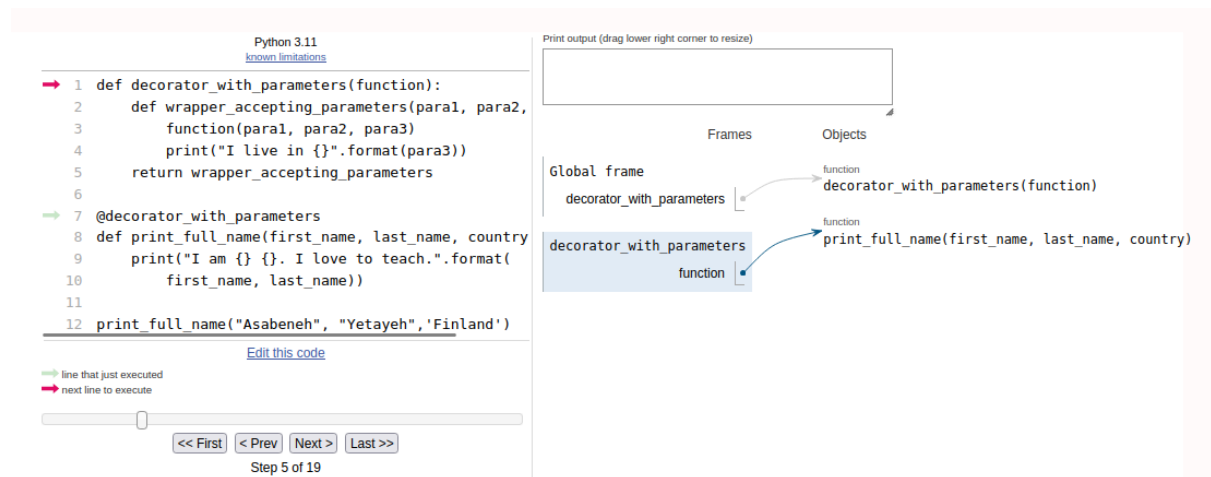
decorator_with_parameters

function

decorator_with_parameters(function)

Step 5: Se abre f1 — Python llama a `decorator_with_parameters(print_full_name)`

Este es el primer momento en que aparece un frame nuevo debajo del Global Frame. El frame se llama `decorator_with_parameters` y contiene una sola variable: `function` → `print_full_name(first_name, last_name, country)`. Dentro de ese frame, Python ejecuta `def wrapper_accepting_parameters(para1, para2, para3):` y crea un nuevo objeto función en el heap con la anotación `[parent=f1]`. La cadena de dependencias en el heap queda así: `wrapper` sabe que su `function` es `print_full_name` original.



Explicación profunda de los Steps 7, 8 y 9 — El **return** y hacia dónde vuelve

Este es el punto más confuso para quienes aprenden decoradores. Se lo explica en detalle.

Step 7 — **return wrapper_accepting_parameters** se está por ejecutar

El visualizador muestra la flecha de "line that just executed" en la línea 2 (**def wrapper_accepting_parameters**) y la flecha de "next line to execute" apuntando a la línea 5 (**return wrapper_accepting_parameters**). En el frame f1 se ven dos variables: **function** y **wrapper_accepting_parameters**, ambas apuntando a sus respectivos objetos función en el heap.

Step 8 — **return** se ejecuta: f1 muestra su Return value

Aquí ocurre el momento crítico. El visualizador muestra por primera vez la etiqueta roja "**Return value**" dentro del frame f1, apuntando al objeto **wrapper_accepting_parameters(para1, para2, para3) [parent=f1]**. ¿Qué significa esto exactamente?

return no destruye f1 todavía. Primero coloca el objeto a retornar en un registro especial del frame —el "Return value"— y señala al intérprete que este frame está listo para cerrarse. El valor que figura en ese Return value es el **objeto función wrapper_accepting_parameters**, no el resultado de llamarla. Se retorna la función en sí misma como dato.

¿A dónde vuelve ese return? Vuelve exactamente al punto del código que invocó a **decorator_with_parameters**. Ese punto es la maquinaria interna del operador **@**, que Python traduce a:

```
print_full_name = decorator_with_parameters(print_full_name)
```

El valor de retorno de `decorator_with_parameters(...)` —que es `wrapper_accepting_parameters`— se asigna al nombre `print_full_name` en el Global Frame. Por eso en el step siguiente `print_full_name` aparece en el Global Frame apuntando al wrapper.

Python 3.11
[known limitations](#)

```
1 def decorator_with_parameters(function):
2     def wrapper_accepting_parameters(para1, para2,
3       function(para1, para2, para3)
4       print("I live in {}".format(para3))
5     return wrapper_accepting_parameters
6
7 @decorator_with_parameters
8 def print_full_name(first_name, last_name, country)
9     print("I am {} {}. I love to teach.".format(
10      first_name, last_name))
11
12 print_full_name("Asabeneh", "Yetayeh", 'Finland')
```

[Edit this code](#)

line that just executed
next line to execute

<< First < Prev Next > Last >>

Step 7 of 19

NEW: teachers get [free access](#) to ad-free/AI-free mode

Print output (drag lower right corner to resize)

Frames

Global frame

decorator_with_parameters

f1: decorator_with_parameters

function

wrapper_accepting_parameters

Objects

function decorator_with_parameters(function)

function print_full_name(first_name, last_name, country)

function wrapper_accepting_parameters(para1, para2, para3) [parent=f1]

Python 3.11
[known limitations](#)

```
1 def decorator_with_parameters(function):
2     def wrapper_accepting_parameters(para1, para2,
3       function(para1, para2, para3)
4       print("I live in {}".format(para3))
5     return wrapper_accepting_parameters
6
7 @decorator_with_parameters
8 def print_full_name(first_name, last_name, country)
9     print("I am {} {}. I love to teach.".format(
10      first_name, last_name))
11
12 print_full_name("Asabeneh", "Yetayeh", 'Finland')
```

[Edit this code](#)

line that just executed
next line to execute

<< First < Prev Next > Last >>

Step 8 of 19

NEW: teachers get [free access](#) to ad-free/AI-free mode

Print output (drag lower right corner to resize)

Frames

Global frame

decorator_with_parameters

f1: decorator_with_parameters

function

wrapper_accepting_parameters

Return value

Objects

function decorator_with_parameters(function)

function print_full_name(first_name, last_name, country)

function wrapper_accepting_parameters(para1, para2, para3) [parent=f1]

Step 9 — f1 desaparece, `print_full_name` en el Global Frame ahora apunta al wrapper

El frame f1 fue destruido. El visualizador muestra que en el Global Frame `print_full_name` ya apunta a `wrapper_accepting_parameters(para1, para2, para3) [parent=f1]`. El objeto función original `print_full_name(first_name, last_name, country)` sigue vivo en el heap —porque el closure del wrapper lo referencia a través de `function`— pero ya no es directamente accesible por su nombre desde el Global Frame.

Python 3.11
[known limitations](#)

```

1 def decorator_with_parameters(function):
2     def wrapper_accepting_parameters(para1, para2,
3         function(para1, para2, para3)
4         print("I live in {}".format(para3))
5     return wrapper_accepting_parameters
6
7 @decorator_with_parameters
8 def print_full_name(first_name, last_name, country)
9     print("I am {} {}. I love to teach.".format(
10         first_name, last_name))
11
12 print_full_name("Asabeneh", "Yetayeh", 'Finland')

```

Edit this code

line that just executed

next line to execute

<< First

< Prev

Next >

Last >>

Step 9 of 19

NEW: teachers get [free access](#) to ad-free/AI-free mode

Print output (drag lower right corner to resize)

Frames

Objects

Global frame

decorator_with_parameters

f1: decorator_with_parameters

function

decorator_with_parameters(function)

print_full_name

function

print_full_name(first_name, last_name, country)

wrapper_accepting_parameters

function

wrapper_accepting_parameters(para1, para2, para3) [parent=f1]

Return value

Fase 2 — Ejecución: propagación de argumentos a través del wrapper

Steps 10–12: `print_full_name("Asabeneh", "Yetayeh", "Finland")` se ejecuta

Python busca `print_full_name` en el Global Frame, encuentra `wrapper_accepting_parameters`, y lo llama con los tres argumentos. Se abre el frame `wrapper_accepting_parameters [parent=f1]`. El visualizador muestra ese frame con tres variables ya pobladas: `para1 = "Asabeneh"`, `para2 = "Yetayeh"`, `para3 = "Finland"`. En el step 11 y 12 el frame del wrapper ya es visible con sus valores, y en el heap el objeto `wrapper_accepting_parameters` muestra las referencias del closure.

Python 3.11
[known limitations](#)

```

1 def decorator_with_parameters(function):
2     def wrapper_accepting_parameters(para1, para2,
3         function(para1, para2, para3)
4         print("I live in {}".format(para3))
5     return wrapper_accepting_parameters
6
7 @decorator_with_parameters
8 def print_full_name(first_name, last_name, country)
9     print("I am {} {}. I love to teach.".format(
10         first_name, last_name))
11
12 print_full_name("Asabeneh", "Yetayeh", 'Finland')

```

Edit this code

line that just executed

next line to execute

<< First

< Prev

Next >

Last >>

Step 10 of 19

NEW: teachers get [free access](#) to ad-free/AI-free mode

Print output (drag lower right corner to resize)

Frames

Objects

Global frame

decorator_with_parameters

print_full_name

f1: decorator_with_parameters

function

decorator_with_parameters(function)

print_full_name

function

print_full_name(first_name, last_name, country)

wrapper_accepting_parameters

function

wrapper_accepting_parameters(para1, para2, para3) [parent=f1]

Return value

Python 3.11
[known limitations](#)

```

1 def decorator_with_parameters(function):
2     def wrapper_accepting_parameters(para1, para2,
3       function(para1, para2, para3)
4       print("I live in {}".format(para3))
5       return wrapper_accepting_parameters
6
7 @decorator_with_parameters
8 def print_full_name(first_name, last_name, country)
9   print("I am {} {}. I love to teach.".format(
10     first_name, last_name))
11
12 print_full_name("Asabeneh", "Yetayeh", 'Finland')

```

Step 11 of 19

NEW: teachers get [free access](#) to ad-free/AI-free mode

Print output (drag lower right corner to resize)

Frames

Global frame

- decorator_with_parameters
- print_full_name
- f1: decorator_with_parameters
- wrapper_accepting_parameters

Objects

- function decorator_with_parameters(function)
- function print_full_name(first_name, last_name, country)
- function wrapper_accepting_parameters(para1, para2, para3) [parent=f1]
- Return value

wrapper_accepting_parameters [parent=f1]

para1	"Asabeneh"
para2	"Yetayeh"
para3	"Finland"

Python 3.11
[known limitations](#)

```

1 def decorator_with_parameters(function):
2     def wrapper_accepting_parameters(para1, para2,
3       function(para1, para2, para3)
4       print("I live in {}".format(para3))
5       return wrapper_accepting_parameters
6
7 @decorator_with_parameters
8 def print_full_name(first_name, last_name, country)
9   print("I am {} {}. I love to teach.".format(
10     first_name, last_name))
11
12 print_full_name("Asabeneh", "Yetayeh", 'Finland')

```

Step 12 of 19

NEW: teachers get [free access](#) to ad-free/AI-free mode

Print output (drag lower right corner to resize)

Frames

Global frame

- decorator_with_parameters
- print_full_name
- f1: decorator_with_parameters
- wrapper_accepting_parameters

Objects

- function decorator_with_parameters(function)
- function print_full_name(first_name, last_name, country)
- function wrapper_accepting_parameters(para1, para2, para3) [parent=f1]
- Return value

wrapper_accepting_parameters [parent=f1]

para1	"Asabeneh"
para2	"Yetayeh"
para3	"Finland"

Steps 13–16: Se abre el frame de `print_full_name` original — dos frames activos

Este es el momento de máxima profundidad de la pila. El visualizador muestra simultáneamente:

- Global Frame (arriba)
- f1: `decorator_with_parameters` — frame del closure (ya vacío de ejecución, pero sus objetos siguen en el heap)
- `wrapper_accepting_parameters [parent=f1]` — con `para1`, `para2`, `para3`
- `print_full_name` — el frame de la función original, con `first_name = "Asabeneh"`, `last_name = "Yetayeh"`, `country = "Finland"`

Los argumentos viajan de `para1/para2/para3` del wrapper a `first_name/last_name/country` de la función original como una copia posicional directa.

Python 3.11
[known limitations](#)

```

1 def decorator_with_parameters(function):
2     def wrapper_accepting_parameters(para1, para2,
3       function(para1, para2, para3)
4       print("I live in {}".format(para3))
5       return wrapper_accepting_parameters
6
7 @decorator_with_parameters
8 def print_full_name(first_name, last_name, country)
9   print("I am {} {}. I love to teach.".format(
10     first_name, last_name))
11
12 print_full_name("Asabeneh", "Yetayeh", 'Finland')

```

[Edit this code](#)

⇒ line that just executed

➡ next line to execute

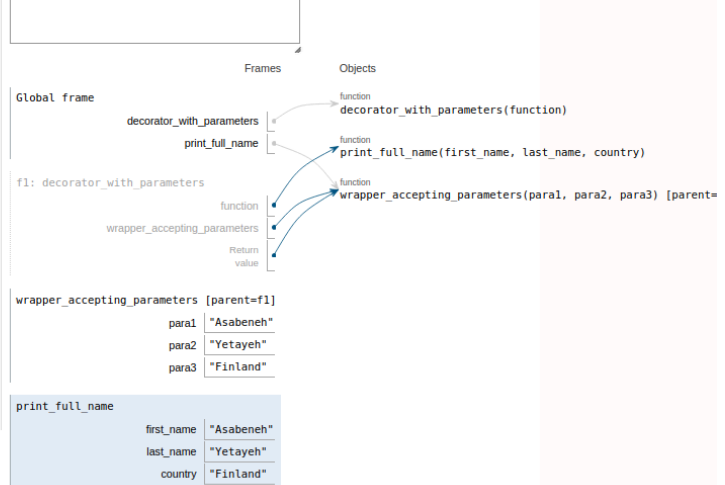
Step 13 of 19

<< First < Prev Next > Last >>

NEW: teachers get [free access](#) to ad-free/AI-free mode

[Move and hide objects \(instructions\)](#)

Print output (drag lower right corner to resize)



Python 3.11
[known limitations](#)

```

1 def decorator_with_parameters(function):
2     def wrapper_accepting_parameters(para1, para2,
3       function(para1, para2, para3)
4       print("I live in {}".format(para3))
5       return wrapper_accepting_parameters
6
7 @decorator_with_parameters
8 def print_full_name(first_name, last_name, country)
9   print("I am {} {}. I love to teach.".format(
10     first_name, last_name))
11
12 print_full_name("Asabeneh", "Yetayeh", 'Finland')

```

[Edit this code](#)

⇒ line that just executed

➡ next line to execute

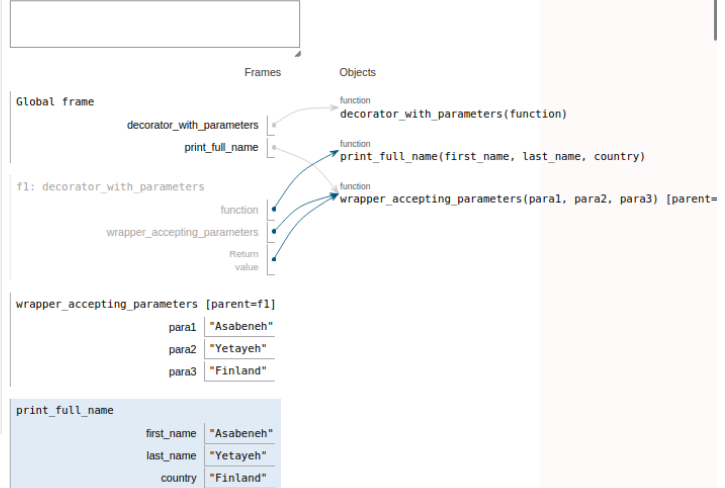
Step 14 of 19

<< First < Prev Next > Last >>

NEW: teachers get [free access](#) to ad-free/AI-free mode

[Move and hide objects \(instructions\)](#)

Print output (drag lower right corner to resize)



Python 3.11
[known limitations](#)

```

1 def decorator_with_parameters(function):
2     def wrapper_accepting_parameters(para1, para2,
3       function(para1, para2, para3)
4       print("I live in {}".format(para3))
5       return wrapper_accepting_parameters
6
7 @decorator_with_parameters
8 def print_full_name(first_name, last_name, country)
9   print("I am {} {}. I love to teach.".format(
10     first_name, last_name))
11
12 print_full_name("Asabeneh", "Yetayeh", 'Finland')

```

[Edit this code](#)

⇒ line that just executed

➡ next line to execute

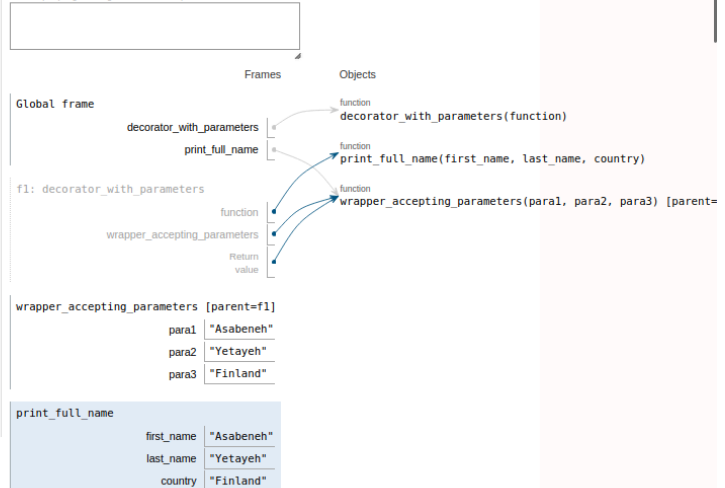
Step 15 of 19

<< First < Prev Next > Last >>

NEW: teachers get [free access](#) to ad-free/AI-free mode

[Move and hide objects \(instructions\)](#)

Print output (drag lower right corner to resize)



The screenshot shows a Python IDE with a code editor on the left and a variable inspector on the right. The code editor displays a decorator function and its application. The variable inspector shows the state of the program at a specific step, including the global frame, the decorator's frame, and the wrapper's frame. The wrapper's frame shows the parameters passed to it, and the variable inspector at the bottom shows the current values of the variables in the global frame.

```

Python 3.11
known limitations

1 def decorator_with_parameters(function):
2     def wrapper_accepting_parameters(para1, para2,
3     function(para1, para2, para3)
4     print("I live in {}".format(para3))
5     return wrapper_accepting_parameters
6
7 @decorator_with_parameters
8 def print_full_name(first_name, last_name, country)
9     print("I am {} {}. I love to teach.".format(
10         first_name, last_name))
11
12 print_full_name("Asabeneh", "Yetayeh", 'Finland')
  
```

Print output (drag lower right corner to resize)

Global frame

- decorator_with_parameters
- print_full_name

f1: decorator_with_parameters

- function
- wrapper_accepting_parameters

wrapper_accepting_parameters (parent=f1)

- para1 "Asabeneh"
- para2 "Yetayeh"
- para3 "Finland"

print_full_name

first_name	"Asabeneh"
last_name	"Yetayeh"
country	"Finland"

Step 17 — `print_full_name` retorna **None**, el wrapper continúa

El frame de `print_full_name` original muestra su **Return value = None** (visible en rojo en el visualizador). Esto ocurre porque la función original no tiene `return` explícito. Ese **None** vuelve al wrapper —pero el wrapper lo descarta completamente: la línea `function(para1, para2, para3)` no guarda el resultado en ninguna variable. El stdout ya muestra la primera línea: `I am Asabeneh Yetayeh. I love to teach.`

Step 18 — El wrapper ejecuta su lógica propia: `print("I live in {}".format(para3))`

Con el frame de `print_full_name` original ya destruido, el wrapper continúa con su siguiente instrucción. Accede a `para3` —que sigue vivo como variable local del frame del wrapper— y ejecuta el `print`. El stdout muestra ahora ambas líneas completas.

Step 19 — El wrapper retorna **None** al Global Frame, ejecución finalizada

El wrapper no tiene `return` explícito, así que su **Return value = None** también. Ese **None** vuelve a la línea `print_full_name(...)` del módulo principal, donde nadie lo captura. La ejecución termina. El visualizador muestra "Done running (19 steps)" y el stdout final con las dos líneas.

```
Python 3.11
known limitations

1 def decorator_with_parameters(function):
2     def wrapper_accepting_parameters(para1, para2, para3):
3         function(para1, para2, para3)
4         print("I live in {}".format(para3))
5     return wrapper_accepting_parameters
6
7 @decorator_with_parameters
8 def print_full_name(first_name, last_name, country):
9     print("I am {} {}. I love to teach.".format(
10         first_name, last_name))
11
12 print_full_name("Asabeneh", "Yetayeh", 'Finland')
```

==> line that just executed
-> next line to execute

Step 17 of 19

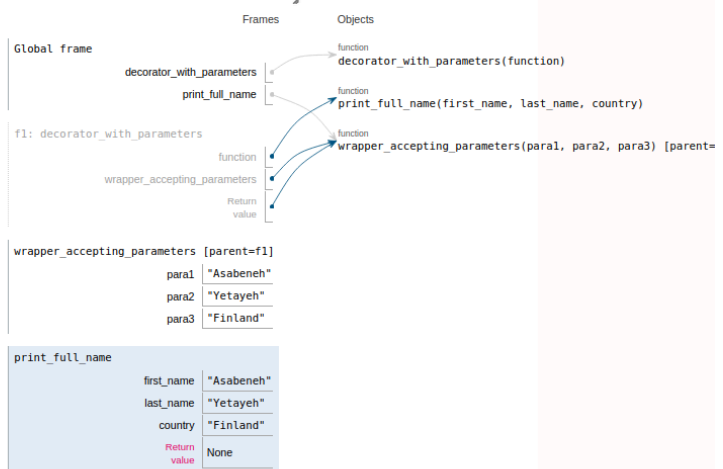
<< First < Prev Next > Last >>

NEW: teachers get [free access](#) to ad-free/AI-free mode

[Move and hide objects \(instructions\)](#)

Print output (drag lower right corner to resize)

I am Asabeneh Yetayeh. I love to teach.



```
Python 3.11
known limitations

1 def decorator_with_parameters(function):
2     def wrapper_accepting_parameters(para1, para2, para3):
3         function(para1, para2, para3)
4         print("I live in {}".format(para3))
5     return wrapper_accepting_parameters
6
7 @decorator_with_parameters
8 def print_full_name(first_name, last_name, country):
9     print("I am {} {}. I love to teach.".format(
10         first_name, last_name))
11
12 print_full_name("Asabeneh", "Yetayeh", 'Finland')
```

==> line that just executed
-> next line to execute

Step 18 of 19

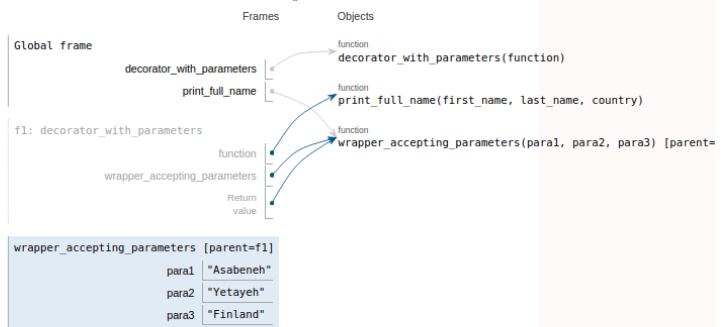
<< First < Prev Next > Last >>

NEW: teachers get [free access](#) to ad-free/AI-free mode

[Move and hide objects \(instructions\)](#)

Print output (drag lower right corner to resize)

I am Asabeneh Yetayeh. I love to teach.



👋 Greetings, human! 🤖

I'm a **new experimental AI Tutor** ready to help you. Your code and visualization will be automatically sent to me, so **do not copy-paste them** into your question.

```
Python 3.11
known limitations

1 def decorator_with_parameters(function):
2   def wrapper_accepting_parameters(para1, para2,
3     function(para1, para2, para3)
4     print("I live in {}".format(para3))
5     return wrapper_accepting_parameters
6
7 @decorator_with_parameters
8 def print_full_name(first_name, last_name, country)
9   print("I am {} {}. I love to teach.".format(
10     first_name, last_name))
11
12 print_full_name("Asabeneh", "Yetayeh", 'Finland')
```

[Edit this code](#)

→ line that just executed
→ next line to execute

<< First < Prev Next > Last >>

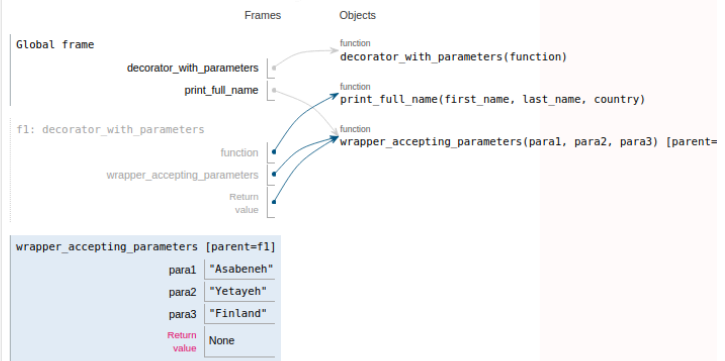
Step 19 of 19

NEW: teachers get [free access](#) to ad-free/AI-free mode

[Move and hide objects \(instructions\)](#)

Print output (drag lower right corner to resize)

```
I am Asabeneh Yetayeh. I love to teach.
I live in Finland
```



👋 Greetings, human! 🤖

I'm a **new experimental AI Tutor** ready to help you. Your code and visualization will be automatically sent to me, so **do not copy-paste them** into your question.

```
Python 3.11
known limitations

1 def decorator_with_parameters(function):
2   def wrapper_accepting_parameters(para1, para2,
3     function(para1, para2, para3)
4     print("I live in {}".format(para3))
5     return wrapper_accepting_parameters
6
7 @decorator_with_parameters
8 def print_full_name(first_name, last_name, country)
9   print("I am {} {}. I love to teach.".format(
10     first_name, last_name))
11
12 print_full_name("Asabeneh", "Yetayeh", 'Finland')
```

[Edit this code](#)

→ line that just executed
→ next line to execute

<< First < Prev Next > Last >>

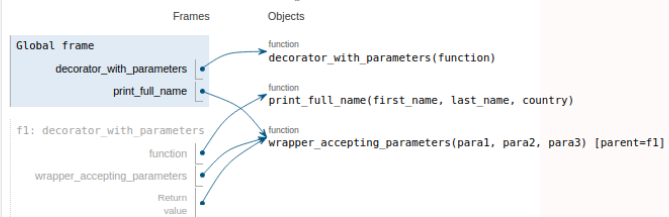
Done running (19 steps)

NEW: teachers get [free access](#) to ad-free/AI-free mode

[Move and hide objects \(instructions\)](#)

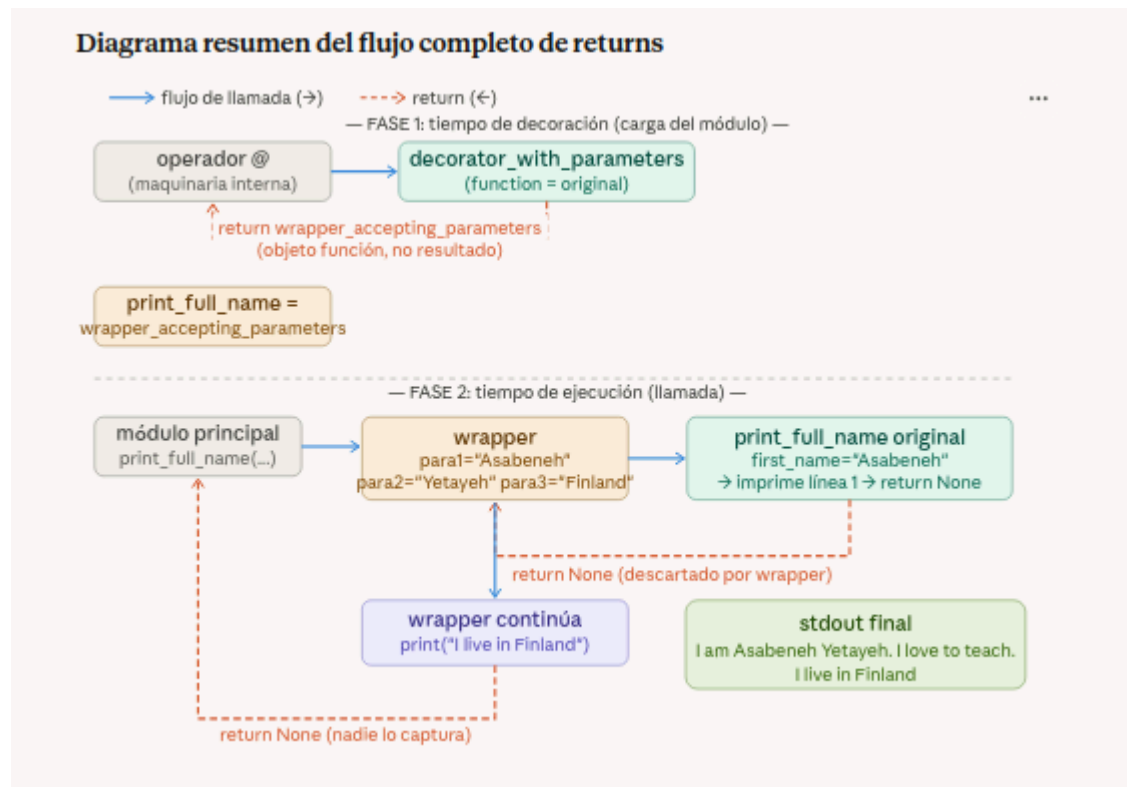
Print output (drag lower right corner to resize)

```
I am Asabeneh Yetayeh. I love to teach.
I live in Finland
```



👋 Greetings, human! 🤖

I'm a **new experimental AI Tutor** ready to help you. Your code and visualization will be automatically sent to me, so **do not copy-paste them** into your question.



Síntesis: los tres **return** del programa y hacia dónde va cada uno

Hay exactamente tres **return** en todo el programa, y ninguno va al mismo lugar:

Return 1 — **return wrapper_accepting_parameters (dentro de **decorator_with_parameters**, steps 7–8).** Este es el que causa confusión. Vuelve al operador **@**, que lo asigna a **print_full_name** en el Global Frame. Lo que se retorna no es un resultado de cómputo sino un **objeto función**. Después de esto, **f1** se destruye.

Return 2 — **return None implícito de **print_full_name** original (step 17).** Vuelve al frame del wrapper, a la línea **function(para1, para2, para3)**. El wrapper no guarda ese **None** en ninguna variable, lo descarta inmediatamente y continúa con la siguiente instrucción: el **print**.

Return 3 — **return None implícito del wrapper (step 19).** Vuelve al módulo principal, a la línea **print_full_name("Asabeneh", "Yetayeh", "Finland")**. Nadie captura ese valor, el programa termina.

